CSS SYSTEMS

CHEAT SHEETS

A Complete Reference for CSS Frameworks, Tooling & Architecture

Tailwind CSS · UnoCSS · Bootstrap · Bulma · Foundation · Styled Components · CSS Modules · Sass/SCSS · PostCSS · Open Props · ITCSS · Migration

48 REFERENCE SHEETS

CONTENTS

UTILITY-FIRST

01 Tailwind Setup & Config
02 Tailwind Layout & Spacing
03 Tailwind Typography & Colors
04 Tailwind States & Responsive
05 Tailwind Customization & Plugins
06 UnoCSS Setup & Presets
07 UnoCSS Utilities & Shortcuts
08 UnoCSS Custom Rules & Theming

COMPONENT FRAMEWORKS

09 Bootstrap Setup & Grid
10 Bootstrap Core Components
11 Bootstrap Utility Classes
12 Bootstrap Forms & Validation
13 Bootstrap Sass Customization
14 Bulma Setup & Columns
15 Bulma Components & Elements
16 Bulma Theming & Modifiers
17 Foundation Setup & XY Grid
18 Foundation UI Components
19 Foundation Sass Settings

CSS-IN-JS

20 Styled Components Setup & Basics
21 Styled Components Props & Dynamic Styles
22 Styled Components Theming & Global
23 Styled Components Advanced Patterns
24 CSS Modules Basics & Scoping
25 CSS Modules Composition & Values
26 CSS Modules Config & Tooling

PREPROCESSORS

27 Sass/SCSS Setup & Syntax
28 Sass/SCSS Variables & Data Types
29 Sass/SCSS Nesting & BEM
30 Sass/SCSS Mixins & Functions
31 Sass/SCSS Module System
32 PostCSS Setup & Config
33 PostCSS Essential Plugins
34 PostCSS Writing Plugins

DESIGN TOKENS & ARCHITECTURE

35 Open Props Setup & Import
36 Open Props Token Reference
37 Open Props Adaptive & Dark Mode
38 ITCSS Architecture & Layers
39 ITCSS Implementation Patterns
40 ITCSS Scaling & Team Workflow

CROSSWALK COMPARISONS

41 Utility Class Comparison
42 Component Framework Comparison
43 CSS-in-JS Comparison
44 Preprocessor Comparison
45 Toolchain Integration Reference
46 Migration & Interop Guide

QUICK REFERENCE

47 Quick Reference — Utility & Component Frameworks
48 Quick Reference — CSS-in-JS, Sass & Architecture

CSYS

Table of Contents

48 sheets · syntax · examples · notes

UTILITY-FIRST 01 – 08

01 Tailwind Setup & Config
02 Tailwind Layout & Spa…
03 Tailwind Typography &…
04 Tailwind States & Resp…
05 Tailwind Customizatio…
06 UnoCSS Setup & Prese…
07 UnoCSS Utilities & Sho…
08 UnoCSS Custom Rules …

COMPONENT FRAMEWORKS 09 – 19

09 Bootstrap Setup & Grid
10 Bootstrap Core Comp…
11 Bootstrap Utility Class…
12 Bootstrap Forms & Vali…
13 Bootstrap Sass Custo…
14 Bulma Setup & Columns
15 Bulma Components & …
16 Bulma Theming & Modi…
17 Foundation Setup & X…
18 Foundation UI Compon…
19 Foundation Sass Settin…

CSS-IN-JS 20 – 26

20 Styled Components Se…
21 Styled Components Pr…
22 Styled Components Th…
23 Styled Components A…
24 CSS Modules Basics & …
25 CSS Modules Composi…
26 CSS Modules Config & …

PREPROCESSORS 27 – 34

27 Sass/SCSS Setup & Sy…
28 Sass/SCSS Variables &…
29 Sass/SCSS Nesting & B…
30 Sass/SCSS Mixins & Fu…
31 Sass/SCSS Module Sys…
32 PostCSS Setup & Config
33 PostCSS Essential Plug…
34 PostCSS Writing Plugins

DESIGN TOKENS & ARCHITECTURE 35 – 40

35 Open Props Setup & I…
36 Open Props Token Ref…
37 Open Props Adaptive …
38 ITCSS Architecture & L…
39 ITCSS Implementation …
40 ITCSS Scaling & Team …

CROSSWALK COMPARISONS 41 – 46

41 Utility Class Comparis…
42 Component Framewor…
43 CSS-in-JS Comparison
44 Preprocessor Compari…
45 Toolchain Integration R…
46 Migration & Interop Gu…

QUICK REFERENCE 47 – 48

47 Quick Reference — Util…
48 Quick Reference — CS…

Tailwind CSS — Setup & Config

tailwind.config.js · @tailwind · content · theme · plugins

WHAT IS TAILWIND CSS?

Tailwind CSS is a utility-first CSS framework that generates a large set of single-purpose classes — `flex`, `pt-4`, `text-center` — that you compose directly in HTML. Unlike component frameworks, Tailwind ships *no pre-built UI*; it provides low-level primitives. At build time, **PurgeCSS / content scanning** removes unused classes so the final bundle is tiny. Tailwind v3+ uses a JIT (Just-In-Time) engine that generates styles on demand.

KEY FACTS

- Current stable: **Tailwind CSS v3** (v4 alpha uses CSS-first config)
- `@tailwind base` injects Preflight (normalize reset)
- `@tailwind components` layer for reusable component classes
- `@tailwind utilities` injects all utility classes
- JIT engine enabled by default in v3 — no `mode: 'jit'` needed
- CDN Play CDN for prototyping only — not for production

INSTALLATION & CONFIG

```
# 1. Install
npm install -D tailwindcss postcss autoprefixer
npx tailwindcss init -p          # creates tailwind.config.js + postcss.config.js

# 2. tailwind.config.js
module.exports = {
  content: [
    './src/**/*.{html,js,jsx,ts,tsx,vue}',
    './public/index.html',
  ],
  theme: {
    extend: {
      colors: { brand: '#5c6bc0' },
      spacing: { '18': '4.5rem' },
      fontFamily: { sans: ['Inter', 'sans-serif'] },
    },
  },
  plugins: [],
}

# 3. CSS entry file (e.g. src/input.css)
@tailwind base;
@tailwind components;
@tailwind utilities;

# 4. Build
npx tailwindcss -i src/input.css -o dist/output.css --watch

# CDN (prototyping only)
<script src="https://cdn.tailwindcss.com"></script>
```

CONFIG EXPLAINED

- → `content` — glob patterns Tailwind scans to find used class names; unused classes are purged
- → `theme.extend` — add to defaults; use bare `theme` to completely replace defaults
- → `colors` — custom color tokens accessible as `bg-brand`, `text-brand`
- → `spacing` — extends the spacing scale; `'18'` → `p-18`, `m-18`
- → `fontFamily.sans` — overrides the default sans stack
- → `plugins` — array of Tailwind plugin functions for custom utilities
- → `-i / -o` — input / output paths for the CLI build
- → `--watch` — rebuilds on file change during development

@TAILWIND DIRECTIVES

DIRECTIVE	INJECTS
`@tailwind base`	Preflight reset + base styles
`@tailwind components`	Component-layer classes
`@tailwind utilities`	All utility classes (JIT-generated)
`@layer base`	Add to base layer (custom resets)
`@layer components`	Define reusable component classes
`@layer utilities`	Define custom utility classes
`@apply`	Use Tailwind classes inside CSS rules

CONFIG TOP-LEVEL KEYS

KEY	PURPOSE
`content`	File globs for class scanning
`theme`	Design tokens (colors, spacing, etc.)
`theme.extend`	Merge with defaults instead of replace
`plugins`	Array of plugin functions
`prefix`	Add prefix to all classes (e.g. `tw-`)
`important`	Force `!important` on all utilities
`darkMode`	`'media'` or `'class'` strategy
`corePlugins`	Disable specific core utilities

FRAMEWORK INTEGRATION

TOOL	CONFIG FILE
`Vite`	`vite.config.js` + PostCSS
`Next.js`	Auto-detected via PostCSS
`Create React App`	PostCSS config in root
`Nuxt`	`@nuxtjs/tailwindcss` module
`Laravel`	Mix / Vite plugin built-in
`Astro`	`@astrojs/tailwind` integration
`SvelteKit`	PostCSS + `svelte.config.js`

⚠ COMMON MISTAKES

- Missing file patterns in `content` — classes used but not scanned get purged in production builds
- Using `theme` instead of `theme.extend` — replaces entire default scale rather than merging
- Forgetting `postcss.config.js` — Tailwind won't process without PostCSS pipeline configured
- Dynamically constructing class names (e.g. `` `text-${color}` ``) — JIT can't scan these; use safelist instead

✓ TIPS

- Use `safelist` in config for dynamically built class names that can't be statically scanned
- Run `npx tailwindcss init --full` to see every default config value
- Use `@apply` sparingly — only for truly reusable component patterns, not one-off styles
- The Tailwind CSS IntelliSense VS Code extension provides autocomplete and hover previews

SUMMARY

- `content` — globs telling Tailwind which files to scan for class names
- `theme.extend` — add tokens without replacing defaults
- `@tailwind base/components/utilities` — three required CSS directives
- `npx tailwindcss -i … -o … --watch` — dev build command

Tailwind CSS — Layout & Spacing

flex · grid · gap · p-* · m-* · w-* · h-* · container

FLEXBOX & GRID EXAMPLES

```
<!-- Flex row, centered, gap -->
<div class="flex items-center justify-between gap-4">
  <span class="flex-1">Left</span>
  <span class="flex-none">Right</span>
</div>

<!-- Flex column, wrapping -->
<div class="flex flex-col flex-wrap gap-2">...</div>

<!-- Grid: 3 equal cols, 4 rows auto -->
<div class="grid grid-cols-3 gap-6">
  <div class="col-span-2">Wide cell</div>
  <div>Cell</div>
</div>

<!-- Responsive grid -->
<div class="grid grid-cols-1 md:grid-cols-2 lg:grid-cols-4 gap-4">
  ...
</div>

<!-- Sizing -->
<div class="w-full h-screen max-w-2xl mx-auto">...</div>
<div class="w-1/2 min-h-64 p-6 m-4">...</div>

<!-- Container with padding -->
<div class="container mx-auto px-4">...</div>
```

CLASS EXPLAINED

- → `flex` — `display: flex`
- → `items-center` — `align-items: center`
- → `justify-between` — `justify-content: space-between`
- → `flex-1` — `flex: 1 1 0%` (grows to fill)
- → `flex-none` — `flex: none` (no grow/shrink)
- → `grid-cols-3` — 3-column grid template
- → `col-span-2` — cell spans 2 columns
- → `gap-4` — `gap: 1rem` (4 × 0.25rem scale)
- → `mx-auto` — `margin-left/right: auto`
- → `max-w-2xl` — `max-width: 42rem`
- → `h-screen` — `height: 100vh`
- → `container` — centered, max-width per breakpoint

SPACING SCALE (P / M / GAP)

CLASS SUFFIX	VALUE	PX EQUIV
`0`	0px	0
`px`	1px	1
`0.5`	0.125rem	2
`1`	0.25rem	4
`2`	0.5rem	8
`3`	0.75rem	12
`4`	1rem	16
`6`	1.5rem	24
`8`	2rem	32
`12`	3rem	48
`16`	4rem	64
`24`	6rem	96

WIDTH / HEIGHT UTILITIES

CLASS	VALUE
`w-full`	100%
`w-screen`	100vw
`w-auto`	auto
`w-1/2`	50%
`w-1/3`	33.333%
`w-64`	16rem (256px)
`max-w-sm`	24rem
`max-w-md`	28rem
`max-w-lg`	32rem
`max-w-xl`	36rem
`max-w-2xl`	42rem
`max-w-prose`	65ch

FLEX & GRID CLASSES

CLASS	CSS
`flex-row`	flex-direction: row
`flex-col`	flex-direction: column
`flex-wrap`	flex-wrap: wrap
`flex-grow`	flex-grow: 1
`flex-shrink-0`	flex-shrink: 0
`order-first`	order: -9999
`grid-cols-{n}`	repeat(n, minmax(0,1fr))
`grid-rows-{n}`	repeat(n, minmax(0,1fr))
`col-span-full`	column span all
`place-items-center`	place-items: center
`self-end`	align-self: flex-end

⚠ COMMON MISTAKES

- › Using `w-full` on a flex child without understanding it interacts with `flex` sizing — combine with `min-w-0` to avoid overflow
- › Forgetting `min-h-0` on flex columns — flex children default `min-height: auto`, causing overflow in nested layouts
- › `container` alone does nothing — always pair with `mx-auto` and `px-*` for centered content

✓ TIPS

- › Use `gap-*` on flex/grid parents instead of margin on children — avoids first/last child edge cases
- › Arbitrary values: `w-[342px]`, `grid-cols-[1fr_2fr]` — use square brackets for one-off values
- › Use `aspect-video` / `aspect-square` for responsive ratio boxes without padding hacks

SUMMARY

- › `flex` / `grid` — activate flex or grid layout on a container
- › `gap-{n}` — spacing between flex/grid children
- › `p-{n}` / `m-{n}` — padding/margin using 0.25rem scale
- › `grid-cols-{n}` — n equal columns; `col-span-{n}` to stretch cells

Tailwind CSS — Typography & Colors

TYPOGRAPHY & COLOR EXAMPLES

```
<!-- Type scale -->
<p class="text-xs">12px</p>    <!-- text-sm=14 text-base=16 -->
<h1 class="text-4xl font-bold tracking-tight">Heading</h1>
<p class="text-lg leading-relaxed">Body text</p>

<!-- Font weight / style -->
<span class="font-thin">100</span>
<span class="font-semibold">600</span>
<span class="font-black italic">900 italic</span>

<!-- Text color with opacity -->
<p class="text-slate-700">Slate 700</p>
<p class="text-blue-600 dark:text-blue-400">Adaptive</p>
<p class="text-black/50">50% opacity black</p>

<!-- Background color -->
<div class="bg-indigo-500">Solid</div>
<div class="bg-gradient-to-r from-purple-500 to-pink-500">Gradient</div>
<div class="bg-white/80 backdrop-blur-sm">Frosted glass</div>

<!-- Text alignment / decoration / transform -->
<p class="text-center truncate uppercase">Truncated</p>
<a class="underline decoration-dotted">Dotted</a>
```

CLASS EXPLAINED

- → `text-{size}` — font-size + matching line-height pair
- → `font-{weight}` — thin(100) to black(900)
- → `tracking-tight` — negative letter-spacing (-0.025em)
- → `leading-relaxed` — line-height: 1.625
- → `text-slate-700` — named palette + shade (50–950)
- → `text-blue-600/75` — color at 75% opacity (v3 syntax)
- → `text-black/50` — shorthand opacity modifier
- → `bg-gradient-to-r` — gradient direction right
- → `from-* / to-*` — gradient stop colors
- → `backdrop-blur-sm` — blur background behind element
- → `truncate` — overflow hidden + ellipsis
- → `uppercase` — text-transform: uppercase

TYPE SCALE

CLASS	SIZE	LINE-HEIGHT
text-xs	12px / 0.75rem	1rem
text-sm	14px / 0.875rem	1.25rem
text-base	16px / 1rem	1.5rem
text-lg	18px / 1.125rem	1.75rem
text-xl	20px / 1.25rem	1.75rem
text-2xl	24px / 1.5rem	2rem
text-3xl	30px / 1.875rem	2.25rem
text-4xl	36px / 2.25rem	2.5rem
text-5xl	48px / 3rem	1
text-6xl	60px / 3.75rem	1

COLOR PALETTES (PARTIAL)

PALETTE	SHADES
slate	50 100 200 … 800 900 950
gray	50 100 200 … 800 900 950
zinc	50 100 200 … 800 900 950
red	50 100 200 … 800 900 950
orange	50 100 200 … 800 900 950
yellow	50 100 200 … 800 900 950
green	50 100 200 … 800 900 950
blue	50 100 200 … 800 900 950
indigo	50 100 200 … 800 900 950
purple	50 100 200 … 800 900 950
pink	50 100 200 … 800 900 950

TEXT UTILITIES

CLASS	EFFECT
text-left	text-align: left
text-center	text-align: center
text-right	text-align: right
text-justify	text-align: justify
truncate	Clamp to one line + ellipsis
line-clamp-{n}	Clamp to n lines
whitespace-nowrap	No line breaks
break-words	Break long words
underline	text-decoration: underline
no-underline	Remove underline
antialiased	-webkit-font-smoothing

⚠ COMMON MISTAKES

- › Using `text-opacity-50` (old API) — v3 uses slash syntax: `text-black/50`
- › Setting `text-{color}` without a shade — `text-blue` alone does nothing; always add shade: `text-blue-500`
- › Gradient requires all three: `bg-gradient-to-r from-X to-Y` — missing `from-` or `to-` renders transparent

✓ TIPS

- › Use `via-{color}` between `from-` and `to-` for three-stop gradients
- › Add `@tailwindcss/typography` plugin for `prose` class — styles rich HTML content automatically
- › Arbitrary colors: `text-[#5c6bc0]` or `bg-[hsl(230,40%,55%)]`

SUMMARY

- › `text-{size}` — sets font-size with paired default line-height
- › `text-{palette}-{shade}` — named color at specific lightness
- › `color/opacity` — slash syntax for color opacity modifiers
- › `bg-gradient-to-{dir} from-X to-Y` — CSS gradient shorthand

Tailwind CSS — States & Responsive

VARIANTS IN ACTION

```
<!-- Pseudo-class variants -->
<button class="bg-blue-500 hover:bg-blue-700 focus:ring-2 active:scale-95">
  Click me
</button>

<!-- Responsive breakpoints (mobile-first) -->
<div class="text-sm md:text-base lg:text-lg xl:text-xl">
  Responsive text
</div>
<div class="block sm:flex lg:grid lg:grid-cols-3">...</div>

<!-- Dark mode (requires darkMode: 'class' in config) -->
<div class="bg-white dark:bg-gray-900 text-black dark:text-white">
  Adapts to dark mode
</div>

<!-- Group: parent state targets children -->
<div class="group">
  <p class="text-gray-400 group-hover:text-white">Reveals on parent hover</p>
</div>

<!-- Peer: sibling state targeting -->
<input type="checkbox" class="peer hidden" id="toggle" />
<label for="toggle" class="peer-checked:text-green-500">Active</label>

<!-- Stacking variants -->
<button class="dark:hover:bg-gray-700 md:focus:ring-4">
  Multiple variants
</button>
```

VARIANTS EXPLAINED

- → `hover:` — applies on `:hover`
- → `focus:` — applies on `:focus`
- → `active:` — applies while pressed
- → `disabled:` — applies when `:disabled`
- → `sm:` — min-width 640px and above
- → `md:` — min-width 768px and above
- → `lg:` — min-width 1024px and above
- → `xl:` — min-width 1280px and above
- → `2xl:` — min-width 1536px and above
- → `dark:` — inside dark mode context
- → `group-hover:` — parent has class `group` and is hovered
- → `peer-checked:` — preceding sibling with `peer` is checked

BREAKPOINTS

PREFIX	MIN-WIDTH	DEVICE TARGET
(none)	0px	All — mobile first
sm:	640px	Small tablet
md:	768px	Tablet
lg:	1024px	Laptop
xl:	1280px	Desktop
2xl:	1536px	Wide desktop
max-sm:	<640px	Max-width variant

STATE VARIANTS

VARIANT	TRIGGER
hover:	Mouse over
focus:	Keyboard / click focus
focus-within:	Child is focused
focus-visible:	Focus via keyboard only
active:	Mouse button held
visited:	Link visited
checked:	Checkbox/radio checked
disabled:	Input disabled
required:	Input required
valid: / invalid:	Form validation state
placeholder:	Placeholder text
first: / last:	:first-child / :last-child

SPECIAL VARIANTS

VARIANT	USE CASE
dark:	Dark mode (media or class)
print:	Print media query
motion-safe:	Respects prefers-reduced-motion
motion-reduce:	Reduced motion active
contrast-more:	High contrast preference
rtl: / ltr:	Text direction
group-{name}:	Named group targeting
peer-{name}:	Named peer targeting
has-[…]:	CSS :has() selector
aria-*:	ARIA attribute variants
data-*:	Data attribute variants

⚠ COMMON MISTAKES

- Using `sm:` to mean "small screens only" — Tailwind is mobile-first; `sm:` means 640px *and up*
- Forgetting `group` class on the parent — `group-hover:` on children does nothing without it
- Setting `dark:` classes without configuring `darkMode: 'class'` — defaults to media query, not class toggle

✓ TIPS

- Stack multiple variants: `dark:md:hover:bg-slate-700` — order is variant:variant:utility
- Use `max-{bp}:` variants for styles that apply *below* a breakpoint
- Named groups: `group/sidebar` + `group-hover/sidebar:opacity-100` for nested group contexts

SUMMARY

- `variant:utility` — prefix any utility with a variant
- `sm: md: lg: xl:` — mobile-first min-width breakpoints
- `group` + `group-hover:` — parent-driven child state
- `peer` + `peer-checked:` — sibling-driven state targeting

Tailwind CSS — Customization & Plugins

theme.extend · @layer · @apply · plugin() · arbitrary values · safelist

CUSTOMIZATION PATTERNS

```
// tailwind.config.js — extending the theme
module.exports = {
  theme: {
    extend: {
      colors: {
        primary: { 50:'#eef2ff', 500:'#6366f1', 900:'#312e81' },
        brand: '#10b981',
      },
      spacing: { '128': '32rem', '144': '36rem' },
      borderRadius: { '4xl': '2rem' },
      screens: { '3xl': '1920px' },
      animation: { 'spin-slow': 'spin 3s linear infinite' },
    },
  },
  safelist: [
    'bg-red-500', 'bg-blue-500',      // always include
    { pattern: /bg-(red|blue)-\d00/ } // regex safelist
  ],
}

// CSS — custom layers and @apply
@layer components {
  .btn-primary {
    @apply px-4 py-2 bg-primary-500 text-white rounded-lg
           hover:bg-primary-600 focus:ring-2 focus:ring-primary-300;
  }
}

@layer utilities {
  .text-balance { text-wrap: balance; }
}

// Plugin — add custom utilities programmatically
const plugin = require('tailwindcss/plugin')
module.exports = {
  plugins: [
    plugin(function({ addUtilities, addComponents, theme }) {
      addUtilities({ '.rotate-y-180': { transform: 'rotateY(180deg)' } })
    }),
    require('@tailwindcss/typography'),
    require('@tailwindcss/forms'),
  ]
}
```

EXPLAINED

- → `theme.extend.colors` — adds tokens; `primary-500` works as `bg-primary-500`
- → `safelist` — forces classes into output even if not found in content scan
- → `pattern: /regex/` — safelist by regex for dynamic classes
- → `@layer components` — groups custom classes at component specificity
- → `@apply` — compose utilities into a CSS rule (use sparingly)
- → `@layer utilities` — adds custom utilities with same specificity as Tailwind's
- → `addUtilities()` — programmatically register utilities in a plugin
- → `addComponents()` — register component classes in a plugin
- → `theme()` — access config values inside plugin

ARBITRARY VALUES

CLASS	OUTPUT
`w-[342px]`	width: 342px
`top-[117px]`	top: 117px
`text-[22px]`	font-size: 22px
`bg-[#bada55]`	background: #bada55
`bg-[url('/img.png')]`	background-image: url(…)
`grid-cols-[1fr_2fr]`	grid-template-columns: 1fr 2fr
`rotate-[17deg]`	rotate: 17deg
`[mask-type:alpha]`	mask-type: alpha (any prop)

OFFICIAL PLUGINS

PACKAGE	ADDS
`@tailwindcss/typography`	`prose` class for rich text
`@tailwindcss/forms`	Form element resets
`@tailwindcss/aspect-ratio`	`aspect-w-*` `aspect-h-*`
`@tailwindcss/container-queries`	`@container` + `@sm:` variants

CSS THEME() FUNCTION

USAGE	RE
`theme('colors.blue.500')`	#3
`theme('spacing.4')`	1re
`theme('fontFamily.sans')`	Fo sta arr
`theme('screens.md')`	76
`theme('borderRadius.lg')`	0.5

⚠ COMMON MISTAKES

- Overusing `@apply` — defeats the purpose of utility-first; prefer component abstractions in JS frameworks
- Forgetting underscores in arbitrary values with spaces — `grid-cols-[1fr_2fr]` not `[1fr 2fr]`
- Not using `safelist` for runtime-generated classes — they won't appear in the production build

✓ TIPS

- Arbitrary properties: `[grid-area:header]` — any CSS property in square brackets
- Use `theme()` inside CSS files to reference config values without duplicating them
- Custom screens: add `'3xl': '1920px'` to get `3xl:` breakpoint variant

SUMMARY

- `theme.extend` — merge new tokens with defaults
- `[arbitrary-value]` — one-off CSS values in square brackets
- `@layer components` + `@apply` — reusable patterns in CSS
- `safelist` — force classes into output that can't be statically scanned

UnoCSS — Setup & Presets

presetUno · presetAttributify · presetIcons · uno.config.ts · on-demand

WHAT IS UNOCSS?

UnoCSS is an atomic CSS engine — not a framework, but an engine that lets you define your own utility rules, or use community presets. It is intentionally **preset-less by default**; all behavior comes from presets you opt into. **presetUno** gives you Tailwind/Windi-compatible classes. Because it generates styles purely on-demand with no pre-scanning step, it is extremely fast. It supports **attributify mode**, where utilities are written as HTML attributes rather than class strings.

KEY FACTS

- Created by Anthony Fu; used heavily in Vite ecosystem
- `presetUno` — Tailwind v3 + Windi CSS compatible utilities
- `presetAttributify` — write utilities as HTML attributes
- `presetIcons` — 100k+ icons as CSS classes via Iconify
- No Tailwind CSS dependency — fully independent engine
- Config file: `uno.config.ts` (TypeScript-first)

INSTALLATION & CONFIG

```
npm install -D unocss

# Vite integration (vite.config.ts)
import UnoCSS from 'unocss/vite'
export default { plugins: [UnoCSS()] }
```

```
// uno.config.ts
import { defineConfig, presetUno, presetAttributify, presetIcons } from 'unocss'

export default defineConfig({
  presets: [
    presetUno(),           // Tailwind-compat utilities
    presetAttributify(),   // attribute mode
    presetIcons({
      scale: 1.2,
      cdn: 'https://esm.sh/',
    }),
  ],
  rules: [
    ['custom-rule', { color: 'red' }],   // static rule
    [/^m-(\d+)$/, ([, d]) => ({ margin: `${d / 4}rem` })], // dynamic
  ],
  shortcuts: {
    'btn': 'px-4 py-2 rounded bg-blue-500 text-white hover:bg-blue-600',
  },
  theme: {
    colors: { primary: '#6366f1' },
  },
})
```

CONFIG EXPLAINED

- → `presetUno()` — activates Tailwind-compatible utility classes
- → `presetAttributify()` — enables HTML attribute mode
- → `presetIcons()` — each Iconify icon becomes a CSS class
- → `rules` — custom static or regex-matched utility definitions
- → `shortcuts` — alias multiple utilities to a single class name
- → `theme` — custom design tokens used in rules and presets
- → `cdn` — load icon data from CDN (browser-only)

AVAILABLE PRESETS

PRESET	PURPOSE
`presetUno`	Tailwind/Windi CSS utilities
`presetAttributify`	Attribute-mode syntax
`presetIcons`	Iconify icon classes
`presetTypography`	Prose class like Tailwind typography
`presetWebFonts`	Google Fonts via config
`presetTagify`	HTML tag as utility class
`presetMini`	Minimal essential utilities only
`presetWind`	Windi CSS-specific extras

ATTRIBUTIFY MODE

HTML ATTRIBUTE	EQUIVALENT CLASS
`text="center xl"`	text-center text-xl
`bg="blue-500"`	bg-blue-500
`p="4"`	p-4
`flex=""`	flex
`hover="bg-blue-600"`	hover:bg-blue-600
`dark="text-white"`	dark:text-white
`sm="flex-col"`	sm:flex-col

FRAMEWORK INTEGRATIONS

TOOL	PACKAGE
`Vite`	unocss/vite (built-in)
`Nuxt`	@unocss/nuxt
`Astro`	unocss/astro
`webpack`	@unocss/webpack
`Next.js`	@unocss/next
`SvelteKit`	unocss/vite
`CDN`	cdn.jsdelivr.net/npm/@unocss

⚠ COMMON MISTAKES

- No presets = no utilities — UnoCSS is an engine; without `presetUno()` none of the common classes work
- Forgetting `import 'virtual:uno.css'` in your app entry — styles won't load without this import
- Mixing attributify and class mode without the `presetAttributify` — attribute syntax is silently ignored

✓ TIPS

- Use `shortcuts` as a component layer — cleaner than `@apply` and works with dynamic rule matching
- UnoCSS Inspector: `http://localhost:5173/__unocss` — see generated CSS in real time
- Use regex rules for design tokens: `[/^color-(.*)$/, ([,c]) => ({ color: `var(--${c})` })]`

SUMMARY

- `presetUno()` — Tailwind-compatible utilities
- `presetAttributify()` — write utilities as HTML attributes
- `rules` — custom static or regex-based utility definitions
- `shortcuts` — alias multiple utilities to one class name

UnoCSS — Utilities & Shortcuts

RULES, SHORTCUTS & VARIANTS

```ts
// uno.config.ts
import { defineConfig, presetUno } from 'unocss'

export default defineConfig({
  presets: [presetUno()],

  // Static shortcuts — simple string alias
  shortcuts: {
    'btn': 'px-4 py-2 rounded bg-blue-500 text-white font-semibold',
    'card': 'rounded-lg shadow-md bg-white p-6',
    'input-base': 'border border-gray-300 rounded px-3 py-2 focus:outline-none',
  },

  // Dynamic shortcuts — regex-matched
  shortcuts: [
    [/^btn-(.*)$/, ([, c]) => `btn bg-${c}-500 hover:bg-${c}-600`],
  ],

  // Custom static rule
  rules: [
    ['flex-center', { display: 'flex', 'align-items': 'center', 'justify-content': 'center' }],
  ],

  // Custom dynamic rule
  rules: [
    [/^grid-area-(.+)$/, ([, area]) => ({ 'grid-area': area })],
    [/^text-stroke-(\d+)$/, ([, w]) => ({ '-webkit-text-stroke': `${w}px` })],
  ],

  // Custom variants
  variants: [
    (matcher) => {
      if (!matcher.startsWith('hocus:')) return matcher
      return { matcher: matcher.slice(6), selector: s => `${s}:hover,${s}:focus` }
    },
  ],

  // Safelist
  safelist: 'prose m-auto flex flex-col items-center'.split(' '),
})
```

EXPLAINED

- → `shortcuts` (object) — static alias; `'btn'` expands to utility string
- → `shortcuts` (array) — regex-matched dynamic shortcuts
- → `rules[0]` (tuple) — static rule: class name → CSS object
- → `rules[1]` (regex) — dynamic rule: regex + function returning CSS
- → `variants` — custom pseudo-class/attribute matchers
- → `matcher` — the class string being matched in variant
- → `selector` — function to rewrite the CSS selector
- → `safelist` — always-included classes regardless of scan

PRESETUNO EXTRA UTILITIES

CLASS	EFFECT
`space-x-{n}`	Horizontal gap between siblings
`space-y-{n}`	Vertical gap between siblings
`divide-x / divide-y`	Borders between siblings
`ring-{n}`	Box-shadow ring
`shadow-{size}`	Drop shadow utilities
`inset-{n}`	top/right/bottom/left shorthand
`aspect-{ratio}`	aspect-ratio shorthand
`sr-only`	Screen-reader only visibility

ICON CLASSES (PRESETICONS)

PATTERN	EXAMPLE
`i-{collection}-{icon}`	i-mdi-home
`i-ph-heart`	Phosphor heart icon
`i-tabler-brand-github`	Tabler GitHub icon
`i-heroicons-solid-star`	Heroicons solid star
`i-logos-vue`	Vue.js logo (colored)
`text-2xl i-mdi-home`	Size via text-* class
`text-red-500 i-mdi-alert`	Color via text-* class

BUILT-IN VARIANTS

VARIANT	CSS SELECTOR
`hover:`	:hover
`focus:`	:focus
`active:`	:active
`dark:`	.dark &
`sm: md: lg:`	min-width breakpoints
`children:`	& > *
`not-first:`	&:not(:first-child)
`[&_a]:`	Arbitrary selector

⚠ COMMON MISTAKES

- Using object `shortcuts` and array `shortcuts` in same config — they must both be in the array form when mixing static and dynamic
- Icon class collection not installed — `presetIcons` requires `@iconify-json/{collection}` package installed
- Writing rules that return strings instead of CSS objects — rules must return `{ property: 'value' }` not CSS strings

✓ TIPS

- Arbitrary variants: `[&:nth-child(3)]:text-red-500` — any selector in square brackets
- Use `children:hover:text-blue-500` to style all child elements on hover of parent
- UnoCSS supports Windi CSS's `space-x-*` and `divide-*` which Tailwind also has — fully compatible

SUMMARY

- `shortcuts` — static or regex-matched class aliases
- `rules` — custom utilities (static tuple or regex function)
- `variants` — custom pseudo-class/variant matchers
- `i-{collection}-{icon}` — icon classes via presetIcons

UnoCSS — Custom Rules & Theming

rules · theme · extendTheme · preflights · layers · on-demand generation

THEMING & ADVANCED CONFIG

```ts
// uno.config.ts — theming
import { defineConfig, presetUno } from 'unocss'

export default defineConfig({
  presets: [presetUno()],

  // Theme — extends/merges with preset theme
  theme: {
    colors: {
      primary: { DEFAULT: '#6366f1', light: '#a5b4fc', dark: '#4338ca' },
      surface: 'hsl(220 14% 96%)',
    },
    breakpoints: {
      xs: '480px', sm: '640px', md: '768px', lg: '1024px',
    },
    fontFamily: { display: '"Cal Sans", sans-serif' },
  },

  // Preflights — inject raw CSS before utilities
  preflights: [
    {
      getCSS: ({ theme }) => `
        :root {
          --color-primary: ${theme.colors.primary.DEFAULT};
          --font-display: ${theme.fontFamily.display};
        }
      `,
    },
  ],

  // Layer ordering
  layers: {
    preflights: -1,
    components: 0,
    default: 1,
    utilities: 2,
    'my-layer': 3,
  },

  // CSS-in-JS rule returning layer-aware styles
  rules: [
    [/^layer-(.+)$/, ([, name], { rawSelector }) => {
      return `@layer ${name} { ${rawSelector} { display: block } }`
    }],
  ],
})
```

EXPLAINED

- → `theme.colors.primary.DEFAULT` — used by `bg-primary` (without shade)
- → `theme.colors.primary.light` — used by `bg-primary-light`
- → `theme.breakpoints` — defines responsive variant widths
- → `preflights` — array of objects with `getCSS()` returning raw CSS strings
- → `getCSS({ theme })` — receives resolved theme for token injection
- → `layers` — controls `@layer` ordering; lower number = earlier
- → `rawSelector` — the generated CSS selector in dynamic rules

THEME STRUCTURE KEYS

KEY	USED BY
colors	bg-* text-* border-* classes
spacing	p-* m-* gap-* w-* h-* classes
breakpoints	sm: md: lg: responsive variants
fontFamily	font-* classes
fontSize	text-{size} classes
fontWeight	font-{weight} classes
lineHeight	leading-* classes
borderRadius	rounded-* classes
boxShadow	shadow-* classes
zIndex	z-* classes

RULE CONTEXT OBJECT

PROPERTY	VALUE
theme	Resolved theme config
rawSelector	Generated CSS selector string
currentSelector	Selector with variants applied
generator	UnoCSS generator instance
constructCSS	Helper to build CSS string
variantHandlers	Active variant transformations

PREFLIGHT PATTERNS

USE CASE	EXAMPLE OUTPUT
CSS custom properties	:root { --color-x: … }
Global resets	*, *::before { box-sizing: border-box }
Font-face declarations	@font-face { src: … }
Base element styles	body { font-family: … }
Layer declarations	@layer base, components, utilities

⚠ COMMON MISTAKES

- Setting theme keys outside of `presetUno` 's expected structure — unknown keys are silently ignored
- Preflights returning undefined — `getCSS()` must always return a string, even an empty one
- Expecting `extendTheme` to work like Tailwind's `theme.extend` — UnoCSS merges at the top-level theme key

✓ TIPS

- Use preflights to inject CSS custom properties derived from theme — keeps tokens in one place
- Name custom layers in `layers` config to control cascade order relative to preset utilities
- Use `theme()` helper in rules to access resolved token values without hardcoding

SUMMARY

- `theme` — design tokens; merged with preset theme defaults
- `preflights` — inject raw CSS at start (resets, custom props)
- `layers` — control `@layer` ordering by name
- `rules` context — access theme, selector, generator in dynamic rules

Bootstrap — Setup & Grid

container · row · col-* · breakpoints · offset-* · order-* · gutter

WHAT IS BOOTSTRAP?

Bootstrap (v5) is the world's most popular CSS framework — a component-first library providing pre-styled UI elements, a 12-column grid, and utility classes. Unlike Tailwind, Bootstrap ships **complete components** (navbars, modals, cards) ready to use. It now requires **no jQuery**; JS components use Vanilla JS bundled as `bootstrap.bundle.min.js`. The grid uses **flexbox** and is mobile-first with six breakpoints.

SETUP OPTIONS

- **CDN:** link CSS + JS bundle in `<head>` and before `</body>`
- **npm:** `npm install bootstrap` — import in JS or Sass
- Current version: **Bootstrap 5.3** with CSS variables and color modes
- `bootstrap.bundle.min.js` includes Popper.js for dropdowns/tooltips
- Sass source: customize variables before importing Bootstrap

GRID SYSTEM

```
<!-- CDN -->
<link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3/dist/css/bootstrap.min.css" rel="stylesheet">
<script src="https://cdn.jsdelivr.net/npm/bootstrap@5.3/dist/js/bootstrap.bundle.min.js"></script>

<!-- Basic 12-column grid -->
<div class="container">
  <div class="row">
    <div class="col-12 col-md-8">Main content</div>
    <div class="col-12 col-md-4">Sidebar</div>
  </div>
</div>

<!-- Auto-width columns -->
<div class="row">
  <div class="col">Equal</div>
  <div class="col">Equal</div>
  <div class="col-6">Double</div>
</div>

<!-- Offset, order, gutter -->
<div class="row g-4">
  <div class="col-md-4 offset-md-2 order-md-last">Shifted</div>
</div>

<!-- Fluid / responsive containers -->
<div class="container-fluid">Full width</div>
<div class="container-md">Fluid until md, then fixed</div>
```

GRID EXPLAINED

- → `container` — centered fixed-width; max-width per breakpoint
- → `container-fluid` — always 100% width
- → `row` — flex container with negative horizontal margins
- → `col` — equal-width auto column
- → `col-{n}` — fixed column count (1–12)
- → `col-{bp}-{n}` — column width at breakpoint and above
- → `offset-{bp}-{n}` — push column right by n columns
- → `order-{n}` — flex order (first, last, 0–5)
- → `g-{n}` — gutters (gap between columns)
- → `gx-{n}` / `gy-{n}` — horizontal / vertical gutter only

BREAKPOINTS

INFIX	MIN-WIDTH	CONTAINER MAX
(none)	0px	100%
sm	576px	540px
md	768px	720px
lg	992px	960px
xl	1200px	1140px
xxl	1400px	1320px

COLUMN PATTERNS

PATTERN	RESULT
`col`	Equal-width (auto)
`col-6`	50% always
`col-sm-6`	50% at sm+
`col-md-4 col-lg-3`	33% at md, 25% at lg
`col-auto`	Content-width column
`w-100`	Force new row break
`row-cols-3`	3 equal cols per row

ALIGNMENT IN GRID

CLASS	EFFECT
`align-items-start`	Row: top-align cols
`align-items-center`	Row: vertically center
`align-items-end`	Row: bottom-align cols
`align-self-start`	Col: self top-align
`justify-content-center`	Row: center cols
`justify-content-between`	Row: space between
`justify-content-end`	Row: right-align cols

⚠ COMMON MISTAKES

- Putting content directly in `.row` without a `.col` — row uses negative margins that will break layout
- Forgetting `.row` inside `.container` — the negative margin compensation only works inside a container
- Using `col-xs-*` — Bootstrap 5 dropped the `xs` infix; unprefixed `col-6` is the xs equivalent

✓ TIPS

- Use `row-cols-{n}` on the row instead of per-child `col-*` for uniform card grids
- `container-{bp}` is fluid below that breakpoint and fixed above — great for full-bleed mobile designs
- Set `$grid-gutter-width` in Sass to change default gutter globally

SUMMARY

- `container > row > col` — the three-layer grid hierarchy
- `col-{bp}-{n}` — 12-column spans at specific breakpoints
- `offset-{bp}-{n}` — push column right by n grid columns
- `g-{n}` — gutter (gap) between grid columns

Bootstrap — Core Components

btn · card · navbar · modal · alert · badge · dropdown · collapse

COMPONENT EXAMPLES

```
<!-- Buttons -->
<button class="btn btn-primary">Primary</button>
<button class="btn btn-outline-secondary btn-sm">Small Outline</button>
<button class="btn btn-danger btn-lg disabled">Disabled</button>

<!-- Card -->
<div class="card shadow">
  <img src="..." class="card-img-top" alt="...">
  <div class="card-body">
    <h5 class="card-title">Title</h5>
    <p class="card-text">Content</p>
    <a href="#" class="btn btn-primary">Go</a>
  </div>
</div>

<!-- Alert with dismiss -->
<div class="alert alert-warning alert-dismissible fade show">
  <strong>Warning!</strong> Message.
  <button type="button" class="btn-close" data-bs-dismiss="alert"></button>
</div>

<!-- Badge / Spinner -->
<span class="badge bg-primary rounded-pill">99+</span>
<div class="spinner-border text-primary" role="status"></div>

<!-- Modal trigger -->
<button data-bs-toggle="modal" data-bs-target="#myModal">Open</button>
<div class="modal fade" id="myModal" tabindex="-1">
  <div class="modal-dialog">
    <div class="modal-content">
      <div class="modal-header">
        <h5 class="modal-title">Title</h5>
        <button class="btn-close" data-bs-dismiss="modal"></button>
      </div>
      <div class="modal-body">Content</div>
      <div class="modal-footer">
        <button class="btn btn-secondary" data-bs-dismiss="modal">Close</button>
      </div>
    </div>
  </div>
</div>
```

COMPONENT NOTES

- → `btn btn-{variant}` — variant: primary secondary success danger warning info light dark link
- → `btn-outline-{variant}` — transparent background, colored border
- → `btn-sm / btn-lg` — small or large button sizing
- → `card-img-top` — image at top of card, rounded-top
- → `alert-dismissible fade show` — animated dismissible alert
- → `data-bs-dismiss="alert"` — JS-powered dismiss via data attr
- → `badge bg-{color}` — inline label with background color
- → `data-bs-toggle="modal"` — triggers modal by target ID
- → `modal-dialog-centered` — vertically centers modal
- → `modal-dialog-scrollable` — scrollable body, fixed header/footer

BUTTON VARIANTS

CLASS	STYLE
`btn-primary`	Blue (brand primary)
`btn-secondary`	Gray
`btn-success`	Green
`btn-danger`	Red
`btn-warning`	Yellow
`btn-info`	Teal
`btn-light`	Light gray
`btn-dark`	Dark
`btn-link`	Looks like a link

JS COMPONENTS

COMPONENT	DATA-BS-TOGGLE
`Modal`	modal
`Dropdown`	dropdown
`Collapse`	collapse
`Offcanvas`	offcanvas
`Tooltip`	tooltip (JS init req.)
`Popover`	popover (JS init req.)
`Tab`	tab
`Toast`	toast (JS init req.)

NAVBAR KEY CLASSES

CLASS	PURPOSE
`navbar`	Base navbar wrapper
`navbar-expand-{bp}`	Horizontal at breakpoint
`navbar-brand`	Logo/brand link
`navbar-toggler`	Hamburger button
`navbar-collapse`	Collapsible nav items
`navbar-nav`	Nav list container
`nav-link active`	Current page link
`navbar-dark bg-dark`	Dark color scheme

⚠ COMMON MISTAKES

- › Forgetting `data-bs-toggle` and `data-bs-target` — modal/dropdown won't work without both attributes
- › Missing `fade show` on alerts — dismissible animation requires both classes
- › Not initializing tooltips/popovers — they require `new bootstrap.Tooltip(el)` JS call; they don't auto-init

✓ TIPS

- › Use `modal-dialog-centered modal-lg` together for a centered large modal
- › Programmatic modal: `const m = new bootstrap.Modal('#myModal'); m.show()`
- › Use `list-group` inside `card` for flush item lists without padding

SUMMARY

- › `btn btn-{variant}` — pre-styled button with color variant
- › `card > card-body > card-title/text` — card structure
- › `data-bs-toggle/target` — JS component activation via HTML
- › `alert alert-{variant} alert-dismissible fade show` — dismissible alert

Bootstrap — Utility Classes

UTILITY EXAMPLES

```
<!-- Display -->
<div class="d-none d-md-block">Hidden on mobile</div>
<div class="d-flex d-md-none">Flex on mobile only</div>

<!-- Flex utilities -->
<div class="d-flex justify-content-between align-items-center gap-3">
  <span class="flex-grow-1">Grows</span>
  <span class="flex-shrink-0">Fixed</span>
</div>

<!-- Spacing: p-{n} m-{n}, 0-5 + auto -->
<div class="p-3 mb-4 mt-0 mx-auto">Spaced</div>
<div class="px-4 py-2">Axis-specific padding</div>

<!-- Text utilities -->
<p class="text-center text-md-start fw-bold fs-4">Heading</p>
<p class="text-truncate text-muted">Muted truncated text</p>

<!-- Colors / backgrounds -->
<div class="bg-primary text-white rounded p-3">Box</div>
<div class="bg-opacity-50 bg-danger">Transparent danger</div>

<!-- Position -->
<div class="position-relative">
  <span class="position-absolute top-0 end-0 badge">New</span>
</div>

<!-- Borders -->
<div class="border border-primary rounded-3 border-2">Bordered</div>
```

UTILITY NOTES

- → `d-{bp}-{value}` — responsive display: none flex block inline grid etc.
- → `justify-content-{val}` — start center end between around evenly
- → `align-items-{val}` — start center end baseline stretch
- → `gap-{n}` — gap between flex/grid children (0–5)
- → `p-{n}` — padding all sides; `px/py/pt/pb/ps/pe` for axes
- → `m-{n}` — margin; `mx-auto` = horizontal centering
- → `fw-bold / fw-semibold` — font-weight utilities
- → `fs-{1-6}` — font-size matching heading scale
- → `top-0 end-0` — absolute position helpers
- → `bg-opacity-{n}` — background opacity (10–75 + 100)

SPACING SCALE

N	REM
0	0
1	0.25r
2	0.5re
3	1rem
4	1.5rer
5	3rem
auto	auto

DISPLAY VALUES

D-{VALUE}	CSS
none	display: none
block	display: block
inline	display: inline
inline-block	display: inline-block
flex	display: flex
inline-flex	display: inline-flex
grid	display: grid
table	display: table

BORDER & SHADOW

CLASS	EFFECT
border	1px border all sides
border-{color}	Border in theme color
border-{n}	Border width 1–5
border-top/bottom/start/end	Single-side border
rounded	border-radius: 0.375rem
rounded-{0-5}	Varying border radius
rounded-circle	50% border radius
shadow-sm / shadow / shadow-lg	Box shadow sizes

⚠ COMMON MISTAKES

- › Using `float-left/right` — Bootstrap 5 uses `float-start/float-end` for RTL support
- › Spacing scale 0–5 is not in px or rem equal steps — 3 is 1rem, 4 is 1.5rem, 5 jumps to 3rem
- › Using `ml-* mr-*` — Bootstrap 5 uses logical properties: `ms-* me-*` (start/end)

✓ TIPS

- › Use `vstack` / `hstack` as shorthand for vertical/horizontal flex stacks
- › `overflow-hidden overflow-auto overflow-scroll` — overflow utilities available in Bootstrap 5.1+
- › Use `ratio ratio-16x9` wrapper for responsive embeds instead of padding hacks

SUMMARY

- › `d-{bp}-{value}` — responsive display control
- › `p-{n} m-{n}` — spacing on 0–5 scale (not linear)
- › `ms-* me-*` — logical inline spacing (start/end)
- › `position-absolute top-0 end-0` — position + placement utilities

Bootstrap — Forms & Validation

form-control · form-select · form-check · input-group · was-validated · is-invalid

FORM EXAMPLES

```
<!-- Floating labels -->
<div class="form-floating mb-3">
  <input type="email" class="form-control" id="email" placeholder="Email">
  <label for="email">Email address</label>
</div>

<!-- Select -->
<select class="form-select">
  <option selected>Choose...</option>
  <option value="1">One</option>
</select>

<!-- Checkboxes / Radios -->
<div class="form-check">
  <input class="form-check-input" type="checkbox" id="cb1">
  <label class="form-check-label" for="cb1">Check me</label>
</div>

<!-- Input group -->
<div class="input-group">
  <span class="input-group-text">@</span>
  <input type="text" class="form-control" placeholder="Username">
  <button class="btn btn-primary">Go</button>
</div>

<!-- Validation -->
<form class="was-validated">
  <input type="text" class="form-control" required>
  <div class="valid-feedback">Looks good!</div>
  <div class="invalid-feedback">Required field.</div>
</form>

<!-- Range / Switch -->
<input type="range" class="form-range">
<div class="form-check form-switch">
  <input class="form-check-input" type="checkbox" role="switch">
</div>
```

FORM NOTES

- → `form-control` — styles text inputs, textarea, file
- → `form-floating` — wrapper for floating label inputs
- → `form-select` — custom-styled select element
- → `form-check` — wrapper for checkbox/radio with label
- → `form-check-input` — styles the input itself
- → `form-switch` — toggle switch variant of checkbox
- → `input-group` — flex container for prepend/append
- → `input-group-text` — text addon beside input
- → `was-validated` — on form: shows valid/invalid states for all
- → `is-invalid / is-valid` — per-input validation state classes
- → `invalid-feedback` — shows error message below input

FORM CONTROL CLASSES

CLASS	ELEMENT
`form-control`	input textarea file
`form-control-sm`	Small input
`form-control-lg`	Large input
`form-control-plaintext`	Read-only, no border
`form-select`	select element
`form-select-sm/lg`	Sized selects
`form-range`	range input
`form-check-input`	checkbox / radio
`form-label`	label styling
`form-text`	Helper text below input

VALIDATION CLASSES

CLASS	USAGE
`was-validated`	On <form>: CSS :valid/:invalid triggers
`is-valid`	On input: force valid state
`is-invalid`	On input: force invalid state
`valid-feedback`	Success message (shows if valid)
`invalid-feedback`	Error message (shows if invalid)
`valid-tooltip`	Success as tooltip bubble
`invalid-tooltip`	Error as tooltip bubble

FORM LAYOUT

CLASS	EFFECT
`mb-3`	Space between form groups
`row g-3`	Grid-based form row
`col-md-6`	Field takes half width at md
`form-floating`	Floating label wrapper
`input-group`	Prepend/append add-ons
`form-check-inline`	Inline checkboxes
`col-form-label`	Label aligned to input in grid

⚠ COMMON MISTAKES

- Forgetting `for` / `id` pairing on floating labels — they must match or the label won't animate
- Using `was-validated` on page load — validation state shows immediately; only add after first submit attempt
- Nesting `input-group` inside `form-floating` — they're incompatible without custom CSS

✓ TIPS

- Use `novalidate` on form + JS validation for full control; add `was-validated` class via JS on submit
- `form-control-color` — styles color picker inputs to match Bootstrap sizing
- Use `row g-3` with `col-*` for multi-column form layouts aligned to the grid

SUMMARY

- `form-control / form-select / form-check-input` — core form element classes
- `form-floating` — wrapper enabling floating label animation
- `input-group` — flex container for prepend/append add-ons
- `was-validated / is-invalid` — two approaches to showing validation state

Bootstrap — Sass Customization

_variables.scss · $primary · $grid-columns · @import · map-merge · dark mode

SASS CUSTOMIZATION

```scss
// Custom Bootstrap setup — src/styles/main.scss

// 1. Override variables BEFORE importing Bootstrap
$primary: #6366f1;
$secondary: #64748b;
$font-family-base: 'Inter', system-ui, sans-serif;
$border-radius: 0.5rem;
$grid-columns: 12;
$grid-gutter-width: 1.5rem;
$spacer: 1rem;

// Custom color map merge
$custom-colors: (
  "brand": #10b981,
  "surface": #f8fafc,
);
$theme-colors: map-merge($theme-colors, $custom-colors);

// 2. Import Bootstrap source
@import "bootstrap/scss/bootstrap";

// 3. Add component overrides after
.btn {
  font-weight: 600;
  letter-spacing: 0.025em;
}

// Dark mode (Bootstrap 5.3+)
[data-bs-theme="dark"] {
  --bs-body-bg: #0f172a;
  --bs-body-color: #e2e8f0;
}

// CSS variables override (no Sass rebuild needed)
:root {
  --bs-primary: #6366f1;
  --bs-primary-rgb: 99, 102, 241;
}
```

EXPLAINED

- → `$primary` — overrides the primary color; must be set BEFORE importing Bootstrap
- → `$theme-colors` — map of color name → hex; extend with `map-merge`
- → `map-merge($theme-colors, $custom)` — adds custom colors without removing defaults
- → `$spacer` — base unit for spacing scale (default 1rem)
- → `$border-radius` — base border-radius for components
- → `@import "bootstrap/scss/bootstrap"` — imports entire framework after overrides
- → `data-bs-theme="dark"` — Bootstrap 5.3 color mode attribute
- → `--bs-primary-rgb` — CSS var needed for opacity utilities to work

KEY SASS VARIABLES

VARIABLE	CONTROLS
`$primary`	Primary color
`$font-family-base`	Body font stack
`$font-size-base`	Base font size (1rem)
`$line-height-base`	Base line height (1.5)
`$border-radius`	Component radius
`$box-shadow`	Default shadow
`$grid-columns`	Column count (12)
`$grid-gutter-width`	Column gap
`$spacer`	Spacing base unit
`$enable-rounded`	Toggle border-radius globally
`$enable-shadows`	Toggle box-shadows globally

SELECTIVE @IMPORT

IMPORT PATH	INCLUDES
`bootstrap/scss/functions`	Required first
`bootstrap/scss/variables`	All default vars
`bootstrap/scss/maps`	Color / size maps
`bootstrap/scss/mixins`	All mixins
`bootstrap/scss/grid`	Grid only
`bootstrap/scss/utilities`	Utility API
`bootstrap/scss/buttons`	Button styles only

CSS VARIABLES (5.3+)

VARIABLE	PURPOSE
`--bs-primary`	Primary color
`--bs-body-bg`	Background color
`--bs-body-color`	Text color
`--bs-link-color`	Link color
`--bs-border-radius`	Component radius
`--bs-font-sans-serif`	Font stack
`--bs-emphasis-color`	High-emphasis text

⚠ COMMON MISTAKES

- Setting Sass variables AFTER `@import "bootstrap"` — they have no effect; variables must precede the import
- Overriding `--bs-primary` CSS var only — opacity utilities still use `--bs-primary-rgb` ; update both
- Using `map-set` instead of `map-merge` — Bootstrap uses `map-merge` for adding to theme maps

✓ TIPS

- Import only needed partials for smaller bundles: functions → variables → maps → mixins → your components
- Use Bootstrap's Utility API (`$utilities` map) to add/remove utility classes in Sass
- Bootstrap 5.3 dark mode: add `data-bs-theme="dark"` to `<html>` or any container

SUMMARY

- `$primary: #hex` BEFORE `@import bootstrap` — override sequence is critical
- `map-merge($theme-colors, $custom)` — extend color map without replacing
- `--bs-*` CSS vars — runtime overrides without Sass rebuild
- `data-bs-theme="dark"` — Bootstrap 5.3 color mode toggle

Bulma — Setup & Columns

columns · column · is-* sizes · is-offset-* · is-multiline · CDN

WHAT IS BULMA?

Bulma is a modern CSS-only framework based on Flexbox. It ships **zero JavaScript** — all interactivity requires your own JS or a companion library. It uses a **simple, readable class naming convention** with `is-*` modifiers and `has-*` helpers. The column system uses fractional widths (`is-half`, `is-one-third`) rather than a numbered 12-column grid. Bulma is fully customizable via Sass variables.

SETUP & FACTS

- **CDN:** `cdn.jsdelivr.net/npm/bulma@0.9.4/css/bulma.min.css`
- **npm:** `npm install bulma` then import Sass partials
- Zero JavaScript — you provide your own interaction logic
- Column system: `columns` container + `column` children
- Responsive: mobile-first with `-mobile` `-tablet` `-desktop` suffixes
- Current version: Bulma 0.9.x (v1.0 released with CSS variables)

COLUMN EXPLAINED

- → `columns` — flex container (the row)
- → `column` — auto-width flex child
- → `is-{n}` — numeric size 1–12 (of 12)
- → `is-half` — 50% width
- → `is-one-third` — 33.33%
- → `is-two-thirds` — 66.67%
- → `is-one-quarter` — 25%
- → `is-three-quarters` — 75%
- → `is-offset-{n}` — push right by n columns
- → `is-multiline` — on columns: allows wrapping to next row
- → `is-gapless` — on columns: removes gaps between columns
- → `is-variable is-{n}` — custom gap size

COLUMN SYSTEM

```
<!-- CDN -->
<link rel="stylesheet" href="https://cdn.jsdelivr.net/npm/bulma@0.9.4/css/bulma.min.css">

<!-- Basic columns (auto-equal widths) -->
<div class="columns">
  <div class="column">First</div>
  <div class="column">Second</div>
  <div class="column">Third</div>
</div>

<!-- Sized columns -->
<div class="columns">
  <div class="column is-two-thirds">Main</div>
  <div class="column">Sidebar</div>
</div>

<!-- Numeric sizes (12-column) -->
<div class="columns">
  <div class="column is-8">8 columns</div>
  <div class="column is-4">4 columns</div>
</div>

<!-- Offset -->
<div class="columns">
  <div class="column is-6 is-offset-3">Centered</div>
</div>

<!-- Multiline wrapping -->
<div class="columns is-multiline">
  <div class="column is-4">Card</div>
  <div class="column is-4">Card</div>
  <div class="column is-4">Card</div>
  <div class="column is-4">Wraps here</div>
</div>
```

COLUMN SIZES

CLASS	WIDTH
`is-1`	8.33%
`is-2`	16.67%
`is-3 / is-one-quarter`	25%
`is-4 / is-one-third`	33.33%
`is-5`	41.67%
`is-6 / is-half`	50%
`is-7`	58.33%
`is-8 / is-two-thirds`	66.67%
`is-9 / is-three-quarters`	75%
`is-10`	83.33%
`is-11`	91.67%
`is-12 / is-full`	100%

RESPONSIVE SUFFIXES

SUFFIX	APPLIES AT
`-mobile`	Up to 768px
`-tablet`	769px to 1023px
`-touch`	Mobile + tablet combined
`-desktop`	1024px to 1215px
`-widescreen`	1216px to 1407px
`-fullhd`	1408px+

COLUMN MODIFIERS

ON .COLUMNS	EFFECT
`is-multiline`	Columns wrap to new rows
`is-gapless`	No space between columns
`is-mobile`	Columns on mobile too
`is-centered`	Center columns horizontally
`is-vcentered`	Vertically center columns
`is-variable is-{n}`	Custom gap (0–8)
`is-reversed`	Reverse column order

⚠ COMMON MISTAKES

- Columns not stacking on mobile — add `is-mobile` to `columns` to keep them side-by-side, or omit it to stack by default
- Forgetting `is-multiline` when column total exceeds 12 — extra columns are hidden without it
- Bulma has no JS — interactive components like modals need `is-active` class toggled via your JS

✓ TIPS

- Use `is-{size}-tablet` for responsive column widths: `is-half-tablet is-full-mobile`
- `is-narrow` on a column makes it only as wide as its content
- Bulma's column system auto-equalizes — no width classes needed for equal-width layouts

SUMMARY

- `columns > column` — two-layer system (no separate row needed)
- `is-{n}` or `is-half/one-third` — fractional or numeric widths
- `is-multiline` — required for wrapping card grids
- `is-{size}-{breakpoint}` — responsive column sizing

Sheet 15

Bulma — Components & Elements

button · card · navbar · hero · section · notification · tag · breadcrumb

COMPONENTS

```
<!-- Buttons -->
<button class="button is-primary is-large">Primary</button>
<button class="button is-outlined is-danger is-rounded">Outlined</button>
<button class="button is-loading">Loading...</button>

<!-- Card -->
<div class="card">
  <div class="card-image"><figure class="image"><img src="..."></figure></div>
  <div class="card-content">
    <p class="title">Card Title</p>
    <p class="content">Content goes here.</p>
  </div>
  <footer class="card-footer">
    <a class="card-footer-item">Save</a>
  </footer>
</div>

<!-- Notification (alert) -->
<div class="notification is-warning">
  <button class="delete"></button>
  Watch out for this!
</div>

<!-- Hero -->
<section class="hero is-primary is-medium">
  <div class="hero-body">
    <p class="title">Hero title</p>
    <p class="subtitle">Subtitle</p>
  </div>
</section>

<!-- Tags -->
<span class="tag is-info is-medium">New</span>
<span class="tag is-success"><span class="icon">✓</span></span>
```

NOTES

- → `is-primary/info/success/warning/danger` — color variants on most components
- → `is-small/medium/large` — size variants
- → `is-outlined` — transparent background, colored border
- → `is-rounded` — full pill border-radius
- → `is-loading` — spinner replaces button content
- → `card-image > figure` — image at top of card
- → `delete` — × button for dismissing notifications/modals (no JS built-in)
- → `hero is-{size}` — small medium large fullheight
- → `hero-body` — centered content inside hero
- → `tag` — inline label; wrap in `tags` for groups

COMPONENT LIST

COMPONENT	KEY CLASS
Button	button is-*
Card	card > card-content
Navbar	navbar > navbar-menu
Hero	hero > hero-body
Modal	modal is-active
Dropdown	dropdown is-active
Notification	notification is-*
Message	message > message-body
Breadcrumb	breadcrumb > li
Pagination	pagination > pagination-list
Tabs	tabs > ul > li.is-active
Panel	panel > panel-block

ELEMENTS

ELEMENT	CLASSES
Box	box
Button	button
Content	content (prose)
Delete	delete
Icon	icon + FA/etc
Image	image is-{ratio}
Notification	notification
Progress	progress is-*
Table	table is-striped
Tag	tag / tags
Title / Subtitle	title / subtitle

NAVBAR STRUCTURE

CLASS	PURPOSE
navbar	Wrapper
navbar-brand	Always-visible left section
navbar-burger	Mobile hamburger
navbar-menu	Collapsible section
navbar-start	Left nav items
navbar-end	Right nav items
navbar-item	Single nav link
is-active	Toggle on navbar-menu (JS)

⚠ COMMON MISTAKES

- Modal visible without `is-active` class — Bulma modals rely entirely on this class being toggled by your JS
- Notification dismiss button does nothing — the `.delete` button requires JS to add `display:none` to the parent
- Navbar hamburger not working — you must write the JS to toggle `is-active` on both the burger and the menu

✓ TIPS

- Use `level` component for horizontal alignment of mixed elements (text + buttons)
- `section` + `container` is Bulma's standard page section pattern
- Use `media` component for comment/avatar layouts — left icon + flexible right content

SUMMARY

- `is-{color}` — color modifier works on most components
- `is-active` — toggled via JS to show modals/dropdowns/menus
- `hero > hero-body` — full-width banner section
- `notification + delete` — dismissible alert (JS handles hide)

Bulma — Theming & Modifiers

$primary · $link · has-text-* · is-primary · is-light · CSS custom properties

BULMA THEMING (SASS)

```scss
// Override Bulma variables BEFORE importing
$primary: #6366f1;
$link: #3b82f6;
$success: #10b981;
$warning: #f59e0b;
$danger: #ef4444;

$family-sans-serif: 'Inter', sans-serif;
$radius: 8px;
$radius-large: 16px;

$navbar-height: 4rem;
$card-shadow: 0 4px 20px rgba(0, 0, 0, 0.1);

// Import Bulma after variables
@import "bulma/bulma";

// Bulma 1.0 CSS variables (no Sass needed)
:root {
  --bulma-primary-h: 231deg;
  --bulma-primary-s: 84%;
  --bulma-primary-l: 60%;
}

// Custom modifier classes following Bulma pattern
.is-shadowed { box-shadow: 0 8px 32px rgba(0,0,0,0.12); }
.has-radius-xl { border-radius: 1.5rem; }
```

THEMING NOTES

- → `$primary` — sets the primary color token; auto-derives dark/light variants
- → `$link` — color for anchor tags and link-styled buttons
- → `$family-sans-serif` — body font stack
- → `$radius` — base border-radius for components
- → `$navbar-height` — changes navbar size and `has-navbar-fixed-top` offset
- → `--bulma-primary-h/s/l` — v1.0 CSS variable system (HSL triplet)
- → `is-*` — modifier pattern; create custom with same pattern
- → `has-*` — helper modifier pattern for text/bg colors

COLOR HELPERS

CLASS	EFFECT
has-text-primary	Text in $primary
has-text-danger	Text in $danger
has-text-grey-light	Muted text
has-text-weight-bold	font-weight: bold
has-background-primary	Background $primary
has-background-light	Light gray background
is-primary	Component primary variant
is-light	Light tint of color
is-dark	Dark shade

KEY SASS VARIABLES

VARIABLE	DEFAULT
$primary	#00d1b2 (teal)
$link	#485fc7 (blue)
$info / $success	Cyan / green
$warning / $danger	Yellow / red
$family-sans-serif	Segoe UI / Oxygen
$size-1 … $size-7	3rem … 0.75rem
$weight-bold	700
$radius	4px
$speed	86ms (transitions)

SPACING HELPERS

CLASS	CSS
m-0 … m-6	margin: 0 … 3rem
p-0 … p-6	padding: 0 … 3rem
mx-auto	margin-x: auto
mt-4 mb-4	Top/bottom margin
is-fullwidth	width: 100%
is-fullheight	height: 100%
is-hidden	display: none
is-invisible	visibility: hidden
is-sr-only	Screen-reader only

⚠ COMMON MISTAKES

- Setting Sass variables after the Bulma `@import` — they must precede it to take effect
- Mixing Bulma 0.9 and 1.0 approaches — 1.0 uses CSS HSL variables; 0.9 uses hex Sass variables
- Using `has-text-black` for body text — use `has-text-grey-darker` for more readable dark text

✓ TIPS

- Import only the Bulma modules you need: `@import "bulma/sass/utilities"` etc.
- Use `$custom-colors` map to add brand colors that automatically get `is-brand` class generated
- Bulma's `is-{color}-light` modifier lightens the background — great for soft badges and alerts

SUMMARY

- `$primary: #hex` before `@import "bulma"` — override sequence
- `has-text-{color}` — text color helper classes
- `has-background-{color}` — background color helpers
- `is-{color}-light` — soft tinted variant of a color

Foundation — Setup & XY Grid

grid-container · grid-x · cell · small-* · medium-* · large-* · gutters

WHAT IS FOUNDATION?

Foundation for Sites (v6) by Zurb is a professional-grade responsive framework with a powerful **XY Grid** system based on CSS Grid and Flexbox. It offers a rich component set including accessible modals, sticky elements, off-canvas navigation, and motion-ui animations. Foundation is highly customizable via Sass and is used heavily in enterprise and agency projects.

SETUP FACTS

- **CDN:** `cdn.jsdelivr.net/npm/foundation-sites@6.8/dist/css/foundation.min.css`
- **npm:** `npm install foundation-sites`
- XY Grid (v6.4+) replaces the old float-based grid
- Breakpoints: small (0), medium (640px), large (1024px), xlarge (1200px), xxlarge (1440px)
- Requires jQuery + Foundation JS for interactive components

XY GRID

```
<!-- Basic XY Grid -->
<div class="grid-container">
  <div class="grid-x grid-margin-x">
    <div class="cell small-12 medium-8">Main</div>
    <div class="cell small-12 medium-4">Sidebar</div>
  </div>
</div>

<!-- Auto cells -->
<div class="grid-x">
  <div class="cell auto">Expands</div>
  <div class="cell shrink">Content width</div>
</div>

<!-- Gutters -->
<div class="grid-x grid-margin-x">...margin gutters...</div>
<div class="grid-x grid-padding-x">...padding gutters...</div>

<!-- Vertical grid -->
<div class="grid-y" style="height:500px">
  <div class="cell small-6">Top half</div>
  <div class="cell small-6">Bottom half</div>
</div>

<!-- Offsets -->
<div class="grid-x">
  <div class="cell medium-6 medium-offset-3">Centered</div>
</div>

<!-- Source order -->
<div class="cell medium-6 medium-order-2">Reordered</div>
```

XY GRID EXPLAINED

- → `grid-container` — centered max-width wrapper
- → `grid-x` — horizontal flex grid row
- → `grid-y` — vertical flex grid (needs explicit height)
- → `cell` — grid child (column)
- → `small-{n}` — width at small+ (1–12)
- → `medium-{n}` — width at medium+ (640px+)
- → `large-{n}` — width at large+ (1024px+)
- → `auto` — cell expands to fill remaining space
- → `shrink` — cell shrinks to content width
- → `grid-margin-x` — margin-based gutters
- → `grid-padding-x` — padding-based gutters
- → `medium-offset-{n}` — push cell right

BREAKPOINTS

NAME	MIN-WIDTH
`small`	0px (all sizes)
`medium`	640px
`large`	1024px
`xlarge`	1200px
`xxlarge`	1440px

COMMON CELL PATTERNS

CLASSES	RESULT
`small-12`	Full width always
`small-6`	50% always
`small-12 medium-6`	Full → half
`small-12 medium-4 large-3`	Full → 1/3 → 1/4
`auto`	Fill remaining space
`shrink`	Content width

GRID ALIGNMENT

CLASS	EFFECT
`align-left`	justify-content: flex-start
`align-center`	justify-content: center
`align-right`	justify-content: flex-end
`align-justify`	space-between
`align-top`	align-items: flex-start
`align-middle`	align-items: center
`align-bottom`	align-items: flex-end
`align-stretch`	align-items: stretch

⚠ COMMON MISTAKES

- Forgetting `grid-container` wrapper — without it, grid margins extend to viewport edge
- Using `grid-margin-x` and `grid-padding-x` together — pick one gutter approach per grid
- Using old `row > columns` syntax — Foundation 6.4+ uses `grid-x > cell` ; the old API is deprecated

✓ TIPS

- Use `grid-container fluid` for full-width layouts without the max-width cap
- `show-for-{bp}` / `hide-for-{bp}` — visibility utility classes for responsive show/hide
- Foundation's Block Grid: `small-up-2 medium-up-3 large-up-4` for equal-column card grids

SUMMARY

- `grid-container > grid-x > cell` — three-layer XY grid
- `small-{n} medium-{n}` — responsive 12-column spans
- `auto / shrink` — flexible cell sizing without fixed columns
- `grid-margin-x / grid-padding-x` — two gutter strategies

Foundation — UI Components

button · callout · card · menu · top-bar · reveal · tabs · accordion

COMPONENT EXAMPLES

```
<!-- Buttons -->
<button class="button primary large">Primary</button>
<button class="button secondary hollow">Hollow</button>
<button class="button success expanded">Full Width</button>

<!-- Callout (alert) -->
<div class="callout warning">
  <h5>Heads Up!</h5>
  <p>This is a warning callout.</p>
  <button class="close-button" data-close>&times;</button>
</div>

<!-- Card -->
<div class="card">
  <div class="card-divider">Card Header</div>
  <img src="...">
  <div class="card-section">
    <h4>Title</h4>
    <p>Content.</p>
  </div>
</div>

<!-- Reveal Modal -->
<button data-open="myModal">Open</button>
<div id="myModal" class="reveal" data-reveal>
  <h1>Modal Title</h1>
  <p>Content.</p>
  <button class="close-button" data-close>&times;</button>
</div>

<!-- Top Bar -->
<div class="top-bar">
  <div class="top-bar-left">
    <ul class="menu"><li class="menu-text">Brand</li></ul>
  </div>
  <div class="top-bar-right">
    <ul class="menu"><li><a href="#">Link</a></li></ul>
  </div>
</div>
```

COMPONENT NOTES

- → `button primary/secondary/success/warnir` — color variants
- → `hollow` — transparent background, colored border/text
- → `expanded` — full-width button
- → `callout` — alert/notification box
- → `data-close` — JS data attribute to close a closable component
- → `close-button` — positioned × dismiss button
- → `card-divider` — header row in card
- → `card-section` — content area inside card
- → `data-reveal` — initializes modal JS plugin
- → `data-open="id"` — trigger to open reveal by ID
- → `top-bar` — horizontal navigation bar
- → `menu` — horizontal/vertical link list

JS COMPONENTS

COMPONENT	DATA-* ATTR
Reveal (Modal)	data-reveal
Accordion	data-accordion
Tabs	data-tabs
Dropdown	data-dropdown
Off-canvas	data-off-canvas
Sticky	data-sticky
Tooltip	data-tooltip
Orbit (Slider)	data-orbit
Equalizer	data-equalizer
Interchange	data-interchange

MENU VARIANTS

CLASS
menu
menu vertical
menu simple
menu align-ri
menu align-ce
menu expanded
menu icon-top
menu nested

BUTTON COLOR VARIANTS

CLASS	COLOR
primary	Brand blue (#1779ba)
secondary	Gray (#767676)
success	Green (#3adb76)
warning	Yellow (#ffae00)
alert	Red (#cc4b37)
hollow	Transparent + border
clear	No border or bg
disabled	Grayed out

⚠ COMMON MISTAKES

- Foundation uses `alert` (not `danger`) for red color — different naming than Bootstrap/Bulma
- JS plugins require initialization: `$(document).foundation()` — won't work without this call
- Reveal modal needs `data-reveal` on the modal AND `data-open="id"` on the trigger — both are required

✓ TIPS

- Use `data-magellan` for sticky section-linked navigation as users scroll
- Foundation's `data-sticky` component handles sticky headers better than CSS `position: sticky` for complex cases
- `button-group` wraps multiple buttons into a connected bar without gaps

SUMMARY

- `button primary/secondary/alert` — color class directly on button
- `callout {color}` — alert box with optional dismiss
- `data-reveal` + `data-open` — modal plugin pair
- `menu` — versatile navigation list with many variants

Foundation — Sass Settings

_settings.scss · $global-* · $breakpoints · $grid-margin-gutters · motion-ui

SASS SETTINGS & SETUP

```scss
// main.scss
// 1. Import settings file (override variables here)
@import 'settings';

// 2. Import Foundation
@import 'foundation-sites/scss/foundation';

// 3. Include components selectively
@include foundation-global-styles;
@include foundation-xy-grid-classes;
@include foundation-typography;
@include foundation-button;
@include foundation-callout;
@include foundation-card;
@include foundation-menu;
@include foundation-top-bar;
@include foundation-reveal;
@include foundation-forms;
@include foundation-visibility-classes;

// _settings.scss — key variables
$global-font-size: 100%;
$global-width: rem-calc(1200);
$global-lineheight: 1.5;
$foundation-palette: (
  primary: #1779ba,
  secondary: #767676,
  success: #3adb76,
  warning: #ffae00,
  alert: #cc4b37,
);
$body-background: #fefefe;
$body-font-color: #0a0a0a;

$breakpoints: (
  small: 0,
  medium: 640px,
  large: 1024px,
  xlarge: 1200px,
  xxlarge: 1440px,
);

$grid-margin-gutters: (
  small: 20px,
  medium: 30px,
);

// rem-calc helper
$global-font-size: 16px; // base for rem-calc()
```

SETTINGS EXPLAINED

- → `_settings.scss` — copy from Foundation source; override here before main import
- → `@include foundation-*` — selective component inclusion for smaller bundles
- → `$global-width` — max-width of grid-container
- → `$foundation-palette` — color map for all theme colors
- → `$breakpoints` — responsive breakpoint map
- → `$grid-margin-gutters` — gutter sizes per breakpoint
- → `rem-calc(px)` — converts px values to rem using $global-font-size
- → `foundation-global-styles` — required base styles mixin

GLOBAL VARIABLES

VARIABLE	DEFAULT
`$global-font-size`	100% (16px)
`$global-width`	75rem (1200px)
`$global-lineheight`	1.5
`$global-radius`	0
`$global-margin`	1rem
`$global-padding`	1rem
`$global-weight-bold`	bold
`$body-font-family`	native font stack
`$body-background`	#fefefe
`$body-font-color`	#0a0a0a

MOTION UI

CLASS	ANIMATION
`slide-in-down`	Slide from top
`slide-in-up`	Slide from bottom
`slide-in-left`	Slide from left
`slide-in-right`	Slide from right
`fade-in`	Opacity 0 → 1
`fade-out`	Opacity 1 → 0
`hinge-in-from-top`	Hinge pivot animation
`spin-in-ccw`	Counter-clockwise spin
`scale-in-up`	Scale from 0 to full

SELECTIVE MIXINS

MIXIN	INCLUDES
`foundation-global-styles`	Required base
`foundation-xy-grid-classes`	XY grid
`foundation-typography`	Type scale
`foundation-forms`	Form elements
`foundation-button`	Button styles
`foundation-callout`	Callout boxes
`foundation-card`	Card component
`foundation-visibility-classes`	Show/hide helpers

⚠ COMMON MISTAKES

- Not importing `_settings.scss` before Foundation — all overrides must come before the Foundation import
- Forgetting `foundation-global-styles` mixin — it's required even if importing other components selectively
- Setting `$global-radius: 0` globally then expecting component-specific radius to work — you must also update those component variables

✓ TIPS

- Use `rem-calc()` everywhere — it keeps all sizes relative to the global font size setting
- Copy the official `_settings.scss` from the Foundation source to your project — it documents every variable
- Use `@include breakpoint(medium)` mixin in Sass for same breakpoints as the grid

SUMMARY

- `_settings.scss` — override variables here before importing Foundation
- `@include foundation-{component}` — selective inclusion for smaller bundles
- `$foundation-palette` — central color map
- `rem-calc(px)` — px to rem conversion using global font size

Styled Components — Setup & Basics

styled.div · css · createGlobalStyle · ThemeProvider · tagged template · keyframes

WHAT ARE STYLED COMPONENTS?

styled-components is a CSS-in-JS library for React that lets you write actual CSS inside JavaScript using **tagged template literals**. Each styled component is a React component with its styles baked in. Styles are **scoped by default** — no class name collisions. At runtime the library generates unique class names and injects a `<style>` tag. It supports **theming**, dynamic props, and **server-side rendering**.

KEY FACTS

- Install: `npm install styled-components`
- v6 is current — requires React 18+ or 16.8+
- Babel plugin or SWC plugin recommended for SSR and display names
- Styles are co-located with components — no separate CSS file
- Supports all CSS including pseudo-classes, media queries, nesting
- Runtime cost: styles injected at runtime (vs build-time CSS)

EXPLAINED

→ `styled.button`…`` — tagged template literal creates a React button component

→ `styled.h1`, `styled.div` etc — one factory per HTML element

→ `&:hover` — self-referencing nesting (like Sass &)

→ `styled(Button)`…`` — extends an existing styled or regular component

→ `createGlobalStyle` — injects global CSS (resets, fonts)

→ `keyframes`…`` — creates a scoped animation name

→ `${"{fadeIn}"}` — interpolates the keyframe into a property

→ `css`…`` — helper for reusable CSS blocks with interpolations

BASICS

```
import styled, { css, createGlobalStyle, keyframes } from 'styled-components'

// Basic styled component
const Button = styled.button`
  background: #6366f1;
  color: white;
  padding: 0.5rem 1rem;
  border: none;
  border-radius: 6px;
  cursor: pointer;

  &:hover { background: #4f46e5; }
  &:disabled { opacity: 0.5; cursor: not-allowed; }
`

// Extending a component
const LargeButton = styled(Button)`
  font-size: 1.25rem;
  padding: 0.75rem 1.5rem;
`

// Any HTML element
const Title = styled.h1`
  font-size: 2rem;
  font-weight: 700;
  color: #0f172a;
`

// Global styles
const GlobalStyle = createGlobalStyle`
  *, *::before, *::after { box-sizing: border-box; }
  body { margin: 0; font-family: 'Inter', sans-serif; }
`

// Keyframe animation
const fadeIn = keyframes`
  from { opacity: 0; transform: translateY(-10px); }
  to   { opacity: 1; transform: translateY(0); }
`
const Animated = styled.div`
  animation: ${fadeIn} 0.3s ease-out;
`

// Usage
function App() {
  return (
    <>
      <GlobalStyle />
      <Title>Hello</Title>
      <Button>Click me</Button>
    </>
  )
}
```

⚠ COMMON MISTAKES

- Defining styled components inside render — recreates the component class on every render, breaking React reconciliation
- Forgetting `<GlobalStyle />` in the tree — it renders nothing visible but must be mounted to inject styles
- Using `styled.div(props => ...)` object syntax without `css` helper — interpolations with props need the tagged template form

✓ TIPS

- Use the Babel plugin (`babel-plugin-styled-components`) for readable class names in dev and SSR support
- Use `.attrs()` to set default HTML attributes: `styled.input.attrs({ type: 'text' })`
- Use `css` helper when building reusable snippet functions that include interpolations

SUMMARY

- `styled.div`css`` — creates a React component with scoped styles
- `styled(Component)`css`` — extends any styled or React component
- `createGlobalStyle` — global CSS (resets, fonts, body styles)
- `keyframes`…`` — scoped animation keyframe definition

Styled Components — Props & Dynamic Styles

PROPS-DRIVEN STYLES

```
import styled, { css } from 'styled-components'

// Inline prop interpolation
const Button = styled.button`
  background: ${props => props.$primary ? '#6366f1' : '#e2e8f0'};
  color: ${props => props.$primary ? 'white' : '#1e293b'};
  padding: ${props => props.$size === 'lg' ? '0.75rem 1.5rem' : '0.5rem 1rem'};
  border-radius: 6px;
  border: none;
`

// Conditional css block (preferred for complex conditions)
const Card = styled.div`
  padding: 1.5rem;
  border-radius: 8px;
  ${props => props.$elevated && css`
    box-shadow: 0 4px 20px rgba(0,0,0,0.12);
    border: none;
  `}
  ${props => props.$variant === 'outlined' && css`
    border: 2px solid #e2e8f0;
    box-shadow: none;
  `}
`

// .attrs() — default HTML attributes + default props
const Input = styled.input.attrs(props => ({
  type: props.type || 'text',
  placeholder: props.placeholder || 'Enter value…',
}))`
  border: 1px solid #cbd5e1;
  border-radius: 4px;
  padding: 0.5rem 0.75rem;
  outline: none;
  &:focus { border-color: #6366f1; box-shadow: 0 0 0 3px rgba(99,102,241,0.2); }
`

// shouldForwardProp — prevent custom props reaching DOM
const Box = styled.div.withConfig({
  shouldForwardProp: prop => !['$gap', '$stack'].includes(prop)
})`
  display: flex;
  flex-direction: ${p => p.$stack ? 'column' : 'row'};
  gap: ${p => p.$gap ?? '1rem'};
`

// Usage
<Button $primary $size="lg">Save</Button>
<Card $elevated $variant="outlined">Content</Card>
<Box $stack $gap="0.5rem">Items</Box>
```

EXPLAINED

- → `${"{props => …}"}` — interpolate any value derived from props
- → `$primary` — transient prop ($ prefix) — not forwarded to DOM in v6+
- → `props.$primary` — access transient prop in interpolation
- → `css`…`` — tag needed when interpolation is inside a conditional block
- → `.attrs(fn)` — merge computed HTML attributes / default props
- → `shouldForwardProp` — return false to block a prop from reaching DOM
- → `$gap ?? '1rem'` — nullish coalescing for prop default values

TRANSIENT PROPS (V5.1+)

PATTERN	DOM FORWARDED?
`$primary ($ prefix)`	No — stays in styled-components only
`primary (no $)`	Yes — reaches HTML element (console warning)
`shouldForwardProp`	Explicit per-prop control
`known HTML attrs`	Always forwarded (className, id, etc.)

INTERPOLATION PATTERNS

PATTERN	USE WHEN
`Inline ternary`	Single property, binary condition
`css`` block`	Multiple properties toggled together
`Theme token`	Value from ThemeProvider
`Object lookup`	Map variant name to value
`.attrs()`	Static/default HTML attributes

CSS SELECTORS IN SC

SYNTAX	TARGETS
`&:hover`	Self on hover
`&:first-child`	Self if first child
`& + &`	Adjacent sibling of same component
`& span`	span inside this component
`${"{OtherComponent}"}`	OtherComponent inside this one
`@media (max-width: …)`	Responsive media query

⚠ COMMON MISTAKES

- Passing custom props without `$` prefix — they reach the DOM, causing React unknown-prop warnings
- Omitting `css`…`` when conditionally returning multi-line CSS blocks — interpolation breaks without the tag
- Using `.attrs()` for styling — attrs sets HTML attributes only; style goes in the template literal

✓ TIPS

- Use object lookup for variants: `const sizes = {"{sm:'0.5rem', lg:'1rem'}"}; padding: ${"{p => sizes[p.$size]}"}`
- Reference another styled component as a selector: `${"{Icon}"} { color: red }` inside another component's styles
- Use `.attrs()` to avoid repetitive prop-passing: set default `type`, `role`, or `aria-*` attributes

SUMMARY

- `${"{p => p.$var}"}` — prop interpolation in template literal
- `$propName` — transient prop; not forwarded to DOM element
- `css`…`` — required tag when returning multi-property conditional blocks
- `.attrs(fn)` — merge default or computed HTML attributes

Styled Components — Theming & Global

ThemeProvider · useTheme · createGlobalStyle · theme tokens · dark mode toggle

THEMEPROVIDER & GLOBAL STYLES

```
import styled, { ThemeProvider, createGlobalStyle, useTheme } from 'styled-components'

// 1. Define theme object
const lightTheme = {
  colors: {
    primary: '#6366f1',
    background: '#ffffff',
    surface: '#f8fafc',
    text: '#0f172a',
    textMuted: '#64748b',
    border: '#e2e8f0',
  },
  space: { sm: '0.5rem', md: '1rem', lg: '1.5rem', xl: '2rem' },
  radius: { sm: '4px', md: '8px', lg: '16px', full: '9999px' },
  shadow: { sm: '0 1px 3px rgba(0,0,0,.1)', lg: '0 10px 40px rgba(0,0,0,.15)' },
}

const darkTheme = {
  ...lightTheme,
  colors: {
    ...lightTheme.colors,
    background: '#0f172a',
    surface: '#1e293b',
    text: '#f1f5f9',
    textMuted: '#94a3b8',
    border: '#334155',
  },
}

// 2. Global styles using theme
const GlobalStyle = createGlobalStyle`
  body {
    background: ${p => p.theme.colors.background};
    color: ${p => p.theme.colors.text};
    font-family: 'Inter', sans-serif;
    transition: background 0.2s, color 0.2s;
  }
`

// 3. Styled component consuming theme
const Card = styled.div`
  background: ${p => p.theme.colors.surface};
  border: 1px solid ${p => p.theme.colors.border};
  border-radius: ${p => p.theme.radius.md};
  padding: ${p => p.theme.space.lg};
  box-shadow: ${p => p.theme.shadow.sm};
`

// 4. useTheme hook in a regular component
function Logo() {
  const theme = useTheme()
  return <svg fill={theme.colors.primary}>...</svg>
}

// 5. Wrap app
function App() {
  const [dark, setDark] = React.useState(false)
  return (
    <ThemeProvider theme={dark ? darkTheme : lightTheme}>
      <GlobalStyle />
      <Card>Content</Card>
    </ThemeProvider>
  )
}
```

THEMING EXPLAINED

- → `ThemeProvider` — context provider; makes theme available to all nested styled components
- → `p.theme.colors.primary` — access theme inside template literal via props
- → `createGlobalStyle` — also receives theme from ThemeProvider
- → `useTheme()` — hook to access theme inside a regular (non-styled) React component
- → `...lightTheme` — spread to inherit and override for dark theme
- → `theme={"{dark ? darkTheme : lightTheme}"}` — swap theme object for dark mode

RECOMMENDED THEME SHAPE

KEY	CONTENTS
`colors`	Named semantic tokens
`space`	Spacing scale (sm/md/lg)
`fontSizes`	Type scale
`fontWeights`	Weight tokens
`radius`	Border-radius tokens
`shadow`	Box-shadow tokens
`breakpoints`	Media query values
`zIndices`	z-index scale

TYPESCRIPT THEME TYPING

STEP	CODE
1. Infer type	`type Theme = typeof lightTheme`
2. Augment module	`declare module 'styled-components' {"{ interface DefaultTheme extends Theme {} }"}`
3. Autocomplete	`p.theme.colors.` shows completions

CREATEGLOBALSTYLE PATTERNS

USE CASE	WHAT TO INCLUDE
CSS reset	* { box-sizing: border-box; margin: 0 }
Font import	@import Google Fonts URL
CSS variables	:root { --color-primary: … }
Body theming	background, color from theme
Selection style	::selection { background: … }
Scrollbar style	::-webkit-scrollbar { … }

⚠ COMMON MISTAKES

- Accessing theme without ThemeProvider — `p.theme` is an empty object; wrap the app in `<ThemeProvider>`
- Calling `useTheme()` outside a ThemeProvider context — returns empty object and components won't style correctly
- Recreating the theme object inside render — creates a new reference every render, causing all themed components to re-render

✓ TIPS

- Define theme as a module-level constant — never inline or inside a component
- Use TypeScript module augmentation for full autocomplete on theme tokens
- Layer `ThemeProvider` — an inner provider overrides the outer, enabling section-level theming

SUMMARY

- `ThemeProvider theme={"{…}"}` — passes theme to all nested styled components
- `p.theme.key` — access theme tokens inside template literals
- `useTheme()` — hook for theme access in regular components
- `createGlobalStyle` — theme-aware global CSS injection

Styled Components — Advanced Patterns

polymorphic as · style objects · TypeScript · ServerStyleSheet · Linaria · zero-runtime

ADVANCED PATTERNS

```
import styled from 'styled-components'

// 1. Polymorphic "as" prop — change underlying element
const Heading = styled.h2`
  font-size: 1.5rem; font-weight: 700;
`
// <Heading as="h1"> renders an h1 with h2 styles
// <Heading as="p">  renders a p with h2 styles

// 2. Targeting another styled component as child
const Icon = styled.svg`width: 1rem; height: 1rem;`

const Button = styled.button`
  display: flex;
  align-items: center;
  gap: 0.5rem;
  ${Icon} { color: currentColor; }        /* targets Icon inside Button */
  &:hover ${Icon} { transform: scale(1.1); }
`

// 3. Object syntax (alternative to template literals)
const Box = styled.div({
  display: 'flex',
  flexDirection: 'column',
  gap: '1rem',
})

// Object with props
const Text = styled.span(props => ({
  color: props.$muted ? '#64748b' : '#0f172a',
  fontSize: props.$size === 'sm' ? '0.875rem' : '1rem',
}))

// 4. Server-Side Rendering (SSR)
import { ServerStyleSheet } from 'styled-components'

// In Express / Next.js custom _document.js:
const sheet = new ServerStyleSheet()
try {
  const html = renderToString(sheet.collectStyles(<App />))
  const styleTags = sheet.getStyleTags()
  // inject styleTags into HTML head
} finally {
  sheet.seal()
}

// 5. StyleSheetManager — inject into shadow DOM or iframe
import { StyleSheetManager } from 'styled-components'
<StyleSheetManager target={shadowRoot}>
  <App />
</StyleSheetManager>
```

EXPLAINED

- → `as="h1"` — renders the styled component as a different HTML element or React component
- → `${"{Icon}"}` inside another component — generates a CSS selector for that component's class
- → `&:hover ${"{Icon}"}` — targets Icon when parent is hovered
- → `styled.div({"{…}"})` — object syntax; camelCase properties, string values
- → `styled.div(props => ({"{…}"})` — object syntax with prop access
- → `ServerStyleSheet` — collects styles for SSR; prevents FOUC
- → `sheet.collectStyles(jsx)` — wraps JSX to intercept style collection
- → `StyleSheetManager` — redirects style injection to a custom target

CSS-IN-JS ALTERNATIVES

LIBRARY	KEY DIFFERENCE
Linaria	Zero-runtime — extracts CSS at build time
Emotion	Similar API; slightly faster; used in MUI
Vanilla Extract	Type-safe, zero-runtime, CSS files
Panda CSS	Atomic CSS-in-JS, build-time extraction
Stitches	Near zero-runtime, variants API
Pigment CSS	MUI's build-time replacement for Emotion

PERFORMANCE CONSIDERATIONS

ISSUE	FIX
Runtime style injection	Use Linaria / Vanilla Extract for zero-runtime
Component recreation	Never define styled() inside render()
SSR FOUC	Use ServerStyleSheet or streaming SSR setup
Large theme object	Split into multiple contexts or use CSS vars
Re-renders on theme change	Memoize theme object reference

TYPESCRIPT TIPS

PATTERN	CODE
Typed props	`styled.div<{"{ $active: boolean }"}>`
Extending HTML attrs	Props extends automatically
Theme types	Augment DefaultTheme interface
Polymorphic typing	Use Emotion's `as` polymorphic helper

⚠ COMMON MISTAKES

- › Using component selector (`${"{OtherSC}"}`) on a non-styled component — only works when the target is a styled-component
- › Forgetting `sheet.seal()` in SSR — causes memory leaks; always call it in a finally block
- › Using `as` without considering accessibility — changing `as="button"` to `as="div"` loses keyboard semantics

✓ TIPS

- › Use object syntax for styles computed from JS variables — cleaner than template literal string interpolation
- › Consider migrating to Linaria or Vanilla Extract if bundle size or runtime performance is a concern
- › Use `StyleSheetManager disableCSSOMInjection` for environments where CSSOM is unavailable (e.g. print)

SUMMARY

- › `as="element"` — render styled component as different HTML/React element
- › `${"{OtherSC}"}` — target another styled component as CSS selector
- › `ServerStyleSheet` — collect styles for SSR hydration
- › Object syntax — alternative to template literals for dynamic styles

CSS Modules — Basics & Scoping

styles.className · :local · :global · camelCase · composes · Vite/webpack loader

WHAT ARE CSS MODULES?

CSS Modules are CSS files where all class names are locally scoped by default. When imported into JavaScript, each class name is transformed to a unique, hashed identifier — eliminating naming collisions globally. The output is **plain CSS** — no runtime cost. CSS Modules work with any CSS syntax and are supported out-of-the-box by Vite, Next.js, and webpack with `css-loader`.

KEY FACTS

- File naming: `Component.module.css` (the `.module` signals the loader)
- Classes become camelCase JS object properties
- All selectors local by default; use `:global()` for exceptions
- Zero runtime — styles extracted to CSS file at build time
- Works with Sass: `Component.module.scss`
- Vite and Next.js support out of the box — no config needed

EXPLAINED

→ `Component.module.css` — the `.module` signals the bundler to scope this file

→ `import styles from './…'` — styles is an object mapping class names to hashed strings

→ `styles.button` — resolves to a unique class like `Button_button__a1b2c`

→ `:global()` — exempts a selector from local scoping

→ `composes: button` — applies all of another local class's rules

→ `composes: … from './other.module.css'` — compose across files

→ `clsx(styles.a, {"{[styles.b]: condition}"})` — conditional class joining utility

CSS MODULES IN PRACTICE

```css
/* Button.module.css */
.button {
  padding: 0.5rem 1rem;
  border-radius: 6px;
  border: none;
  cursor: pointer;
}

.primary {
  background: #6366f1;
  color: white;
}

.primary:hover {
  background: #4f46e5;
}

/* Escape to global scope */
:global(.third-party-class) {
  color: red;
}

/* Compose from another module */
.icon-button {
  composes: button;          /* local compose */
  display: inline-flex;
  align-items: center;
  gap: 0.5rem;
}

/* camelCase recommended for JS access */
.submitButton {
  composes: button primary;
  font-weight: 600;
}
```

```jsx
// Button.jsx
import styles from './Button.module.css'

function Button({ primary, children }) {
  return (
    <button className={`${styles.button} ${primary ? styles.primary : ''}`}>
      {children}
    </button>
  )
}

// With clsx / classnames library (recommended)
import clsx from 'clsx'
<button className={clsx(styles.button, { [styles.primary]: primary })}>
```

⚠ COMMON MISTAKES

- Using kebab-case class names — they become `styles['my-class']` not `styles.myClass`; prefer camelCase
- Applying global overrides without `:global()` — local scoping will hash third-party class names, breaking them
- Forgetting `.module.css` extension — regular `.css` files are global; only `.module.css` is scoped

✓ TIPS

- Use `clsx` or `classnames` library to cleanly combine conditional module classes
- TypeScript: use `typed-css-modules` or `css-modules-typescript-loader` for autocomplete on class names
- Use `composes` to share base styles without duplication — avoids the need for a shared utility class file

SUMMARY

- `Component.module.css` — scoped CSS; all classes are local by default
- `import styles from './…'` — styles is a map of name → hashed class
- `:global()` — opt out of scoping for third-party selectors
- `composes: otherClass` — inherit styles from another local class

CSS Modules — Composition & Values

composes: from · @value · CSS variables · Sass modules · TypeScript typings

COMPOSITION & @VALUE

```
/* colors.module.css — shared design tokens */
@value primary: #6366f1;
@value primaryDark: #4f46e5;
@value success: #10b981;
@value danger: #ef4444;
@value sm: (max-width: 640px);
@value md: (min-width: 641px);

/* Button.module.css */
@value primary, primaryDark from './colors.module.css';

.base {
  padding: 0.5rem 1rem;
  border-radius: 6px;
  border: none;
  font-weight: 500;
  transition: background 0.15s;
}

/* Cross-file composition */
.primaryBtn {
  composes: base;
  composes: focusRing from './shared.module.css';
  background: primary;      /* @value used as CSS value */
  color: white;
}

.primaryBtn:hover {
  background: primaryDark;
}

/* Responsive via @value breakpoint */
@media sm {
  .primaryBtn { width: 100%; }
}

/* Using CSS custom properties alongside modules */
.card {
  composes: base from './shared.module.css';
  background: var(--surface);
  color: var(--text-primary);
  border: 1px solid var(--border);
}

/* Sass module (.module.scss) — full Sass works */
.button {
  @include focus-ring;       /* Sass mixin */
  $pad: 0.5rem;
  padding: $pad calc($pad * 2);
}
```

EXPLAINED

- → `@value name: value` — define a reusable value (color, breakpoint) in a CSS Module
- → `@value name from './file'` — import a value from another module
- → Imported `@value` can be used as a CSS value: `background: primary`
- → `composes: base` — inherits styles from a local class
- → `composes: name from './other.module.css'` — cross-file composition
- → The element gets multiple generated class names — one per composed class
- → `.module.scss` — Sass syntax fully supported; variables and mixins work
- → CSS custom properties (`var(--token)`) complement CSS Modules for runtime theming

COMPOSES VARIANTS

SYNTAX	SOURCE
`composes: foo`	Same file, local class
`composes: foo bar`	Multiple local classes
`composes: foo from './x.module.css'`	Another CSS Module file
`composes: foo from global`	A global CSS class

@VALUE USE CASES

USE CASE	EXAMPLE
Color tokens	`@value primary: #6366f1`
Breakpoints	`@value sm: (max-width: 640px)`
Spacing	`@value gap-md: 1rem`
Font stacks	`@value sans: 'Inter', sans-serif`
Import from	`@value x from './tokens.module.css'`

MODULES VS CSS CUSTOM PROPS

FEATURE	CSS MODULES	CSS VARS
Build time	Yes	Runtime
Dynamic	No	Yes (JS)
Dark mode	Harder	Easy
Scoping	Class hash	Cascade
Combo	Use both — modules for classes, vars for tokens	

⚠ COMMON MISTAKES

- › `composes` must be first in the rule — placing it after other properties causes it to be ignored by some loaders
- › Composing from non-module CSS files — `composes: from './global.css'` only works if that file is also a CSS Module
- › Using `@value` for dynamic values — `@value` is a static build-time substitution; use CSS custom properties for runtime theming

✓ TIPS

- › Combine CSS Modules with CSS custom properties — modules for scoped class names, vars for dynamic theming
- › Create a `tokens.module.css` with all `@value` declarations; import tokens across all feature modules
- › Use `composes: from global` to apply a third-party animation class without losing module scoping elsewhere

SUMMARY

- › `@value name: value` — define reusable tokens in a CSS Module
- › `@value x from './file'` — import tokens across modules
- › `composes: class from './file.module.css'` — cross-file composition
- › CSS vars + CSS Modules — best of both: scoped classes + runtime theming

CSS Modules — Config & Tooling

TOOLING CONFIGURATION

```
// ── webpack — css-loader config ──────────────────────────
module.exports = {
  module: {
    rules: [{
      test: /\.module\.css$/,
      use: ['style-loader', {
        loader: 'css-loader',
        options: {
          modules: {
            localIdentName: '[name]__[local]--[hash:base64:5]',
            // dev:  'Button__primary--a1b2c'
            // prod: '[hash:base64:8]' for smaller output
            exportLocalsConvention: 'camelCaseOnly',
            // 'camelCaseOnly' → styles.myButton (not styles['my-button'])
            mode: 'local', // 'global' | 'pure'
          },
        },
      }],
    }],
  },
}

// ── Vite ─────────────────────────────────────────────────
// vite.config.js
export default {
  css: {
    modules: {
      localsConvention: 'camelCaseOnly',
      generateScopedName: '[name]_[local]_[hash:5]',
      // In dev: descriptive; in prod: short hash
    },
  },
}

// ── Next.js ──────────────────────────────────────────────
// Built-in — no config needed; .module.css auto-scoped

// ── TypeScript typings ───────────────────────────────────
// tsconfig.json — enable path alias for type generation
// Use: css-modules-typescript-loader or
//      typed-css-modules (tcm) CLI

// global.d.ts — manual declaration fallback
declare module '*.module.css' {
  const classes: Record<string, string>
  export default classes
}
declare module '*.module.scss' {
  const classes: Record<string, string>
  export default classes
}
```

CONFIG EXPLAINED

- → `localIdentName` — controls the generated class name format
- → `[name]` — source filename without extension
- → `[local]` — original class name from CSS
- → `[hash:base64:5]` — 5-char content hash for uniqueness
- → `exportLocalsConvention: 'camelCaseOnly'` — only camelCase keys in JS export
- → `mode: 'local'` — all classes local (default); `pure` disallows :global
- → `localsConvention` — Vite equivalent of exportLocalsConvention
- → `Record<string, string>` — TypeScript type for the styles object

LOCALIDENTNAME TOKENS

TOKEN	EXPANDS TO
`[name]`	Source filename (no ext)
`[local]`	Original class name
`[path]`	Relative file path
`[folder]`	Containing folder name
`[hash:base64:N]`	N-char content hash
`[contenthash:N]`	File content hash

FRAMEWORK DEFAULTS

TOOL	CONFIG LOCATION	AUTO?
Vite	vite.config.js css.modules	Yes
Next.js	Built-in	Yes
CRA	Built-in (no eject)	Yes
webpack	css-loader options.modules	Manual
Parcel	Built-in	Yes
Astro	Built-in	Yes

TYPESCRIPT TOOLS

TOOL	HOW IT WORKS
`typed-css-modules (tcm)`	CLI generates .d.ts files
`css-modules-typescript-loader`	webpack loader, generates types
`vite-plugin-dts`	Vite plugin for .d.ts output
`global.d.ts declare`	Fallback; no per-class types
`TypeScript 5.5 CSS types`	Native support planned

⚠ COMMON MISTAKES

- Using short hashes in development — hard to debug; use descriptive names in dev, short hashes only in production
- Not configuring `exportLocalsConvention` — kebab-case classes need bracket access; camelCaseOnly is cleaner
- Using `mode: 'pure'` and then needing `:global` — pure mode disallows global selectors; use `local` instead

✓ TIPS

- Use `[name]__[local]--[hash:5]` in dev for BEM-style readable class names with uniqueness guarantee
- Run `tcm src --watch` alongside dev server to keep TypeScript types in sync with CSS changes
- Pair CSS Modules with PostCSS plugins — they run before the scoping transformation

SUMMARY

- `localIdentName` — controls generated class name format
- `camelCaseOnly` — dot-notation access in JS (`styles.myClass`)
- Most modern tools (Vite, Next.js) support CSS Modules with zero config
- `tcm` CLI — generate TypeScript types for CSS Module class names

Sass/SCSS — Setup & Syntax

sass · scss · dart sass · .scss vs .sass · CLI · npm · watch

WHAT IS SASS?

Sass (Syntactically Awesome Style Sheets) is the most widely used CSS preprocessor, adding variables, nesting, mixins, functions, and modules to CSS. The modern implementation is **Dart Sass** — the reference implementation, actively maintained. Sass has two syntaxes: **SCSS** (superset of CSS, uses braces) and the indented **.sass** syntax (uses indentation, no braces). SCSS is overwhelmingly preferred in modern projects.

KEY FACTS

- **Dart Sass** is the current standard; LibSass and Ruby Sass are deprecated
- `.scss` — superset of CSS; valid CSS is valid SCSS
- `.sass` — indented syntax; no braces or semicolons
- Install: `npm install -D sass`
- Vite, webpack (sass-loader), and Parcel support Sass natively
- New module system (`@use` / `@forward`) replaced `@import` in Dart Sass

SYNTAX EXPLAINED

- → `$variable` — Sass variable; replaced at compile time
- → `&` — parent selector reference; combines with parent class
- → `&:hover` — generates `.card:hover`
- → `&--modifier` — generates `.card--modifier` (BEM pattern)
- → `#{$var}` — interpolation; inserts variable into selector/property
- → `@use 'sass:math'` — import a Sass built-in module
- → `color.adjust()` — programmatically modify a color
- → `math.div(a, b)` — division (replaces deprecated `a / b`)

SETUP & CORE SYNTAX

```scss
// Install and compile
// npm install -D sass
// npx sass src/main.scss dist/main.css --watch

// Variables (Sass) vs CSS custom properties
$primary: #6366f1;          // Sass variable — compile-time
$font-stack: 'Inter', sans-serif;
$spacing-unit: 1rem;

// CSS custom properties — runtime
:root { --primary: #6366f1; }

// Nesting
.card {
  background: white;
  border-radius: 8px;

  &:hover { box-shadow: 0 4px 20px rgba(0,0,0,.1); }
  &--featured { border: 2px solid $primary; }   // BEM modifier

  .card__title {       // descendent
    font-size: 1.25rem;
    color: $primary;
  }
}

// Parent selector reference
.btn {
  &-primary { background: $primary; }
  &-secondary { background: #64748b; }
  &:focus-visible { outline: 2px solid $primary; }
}

// Interpolation
$side: top;
.box { border-#{$side}: 1px solid red; }  // border-top

// @use — new module system
@use 'sass:math';
@use 'sass:color';
$lighter: color.adjust($primary, $lightness: 20%);
$half: math.div(100%, 2);   // 50%
```

SASS CLI COMMANDS

COMMAND	ACTION
`sass src/main.scss dist/out.css`	Compile once
`sass --watch src/:dist/`	Watch folder
`sass --style=compressed`	Minified output
`sass --no-source-map`	Disable source maps
`sass --load-path=src`	Add import search path

BUILT-IN MODULES (@USE)

MODULE	KEY FUNCTIONS
`sass:math`	div · floor · ceil · round · abs
`sass:color`	adjust · mix · lighten · darken · invert
`sass:string`	quote · unquote · to-upper-case · length
`sass:list`	append · join · length · nth · index
`sass:map`	get · set · merge · keys · values
`sass:selector`	nest · append · parse
`sass:meta`	type-of · call · mixin-exists

SCSS VS .SASS SYNTAX

FEATURE	SCSS	.SASS
Braces	Yes	No (indent)
Semicolons	Yes	No
Valid CSS	Yes (superset)	No
Nesting	{ }	Indentation
Mixins	@mixin / @include	=name / +name
Popularity	Dominant	Rare

⚠ COMMON MISTAKES

- Still using `@import` — it is deprecated in Dart Sass; use `@use` and `@forward` instead
- Using `/` for division — deprecated; use `math.div(a, b)` or `calc(a / b)`
- Installing `node-sass` — it wraps LibSass (deprecated); install `sass` (Dart Sass) instead

✓ TIPS

- Use `sass:color` functions instead of CSS filter tricks for programmatic color manipulation
- Use `--load-path` to avoid long relative import paths: `@use 'tokens'` instead of `@use '../../../tokens'`
- Vite uses Sass automatically when it detects `.scss` — no extra plugin needed, just `npm install -D sass`

SUMMARY

- `$variable` — compile-time variable replaced in output CSS
- `&` — parent selector; enables nesting and BEM patterns
- `@use 'sass:color'` — import built-in module for color functions
- `math.div()` — safe division (replaces deprecated `/` operator)

Sass/SCSS — Variables & Data Types

$var · !default · maps · lists · colors · strings · numbers · booleans

VARIABLES & DATA TYPES

```scss
// ── Numbers ──────────────────────────────────────────
$size: 1rem;
$count: 3;
$ratio: 1.618;
$percent: 75%;

// ── Strings ──────────────────────────────────────────
$family: 'Inter', sans-serif;    // quoted
$selector: button;               // unquoted

// ── Colors ───────────────────────────────────────────
$primary: #6366f1;
$primary: hsl(239, 84%, 67%);
$primary: rgb(99, 102, 241);

// ── Booleans / Null ──────────────────────────────────
$enable-shadows: true;
$empty: null;                    // omits property if used

// ── Lists ────────────────────────────────────────────
$margins: 10px 20px 30px;        // space-separated
$fonts: ('Inter', 'Roboto');     // comma-separated

@use 'sass:list';
$first: list.nth($margins, 1);   // 10px
$len:   list.length($fonts);     // 2

// ── Maps ─────────────────────────────────────────────
@use 'sass:map';

$colors: (
  'primary': #6366f1,
  'secondary': #64748b,
  'success': #10b981,
);

$primary: map.get($colors, 'primary');    // #6366f1
$updated: map.set($colors, 'danger', #ef4444);
$merged:  map.merge($colors, ('info': #0ea5e9));

// Iterate map
@each $name, $value in $colors {
  .bg-#{$name} { background-color: $value; }
}

// !default — set only if not already defined
$grid-cols: 12 !default;   // won't override if $grid-cols already set
```

EXPLAINED

- → `$variable: value` — lexically scoped Sass variable
- → `!default` — sets value only if variable is undefined or null; used in library partials
- → `null` — when used as a property value, the property is omitted entirely
- → `list.nth($list, 1)` — 1-indexed list access
- → `map.get($map, 'key')` — retrieve a value from a map
- → `map.merge($a, $b)` — shallow merge two maps
- → `@each $k, $v in $map` — iterate map key-value pairs
- → `.bg-#{$name}` — interpolate variable into class name

SASS DATA TYPES

TYPE	EXAMPLE
`Number`	1 · 1.5 · 10px · 75% · 1em
`String`	'Inter' · bold · none
`Color`	#fff · red · hsl(0,0%,100%)
`Boolean`	true · false
`Null`	null
`List`	10px 20px · ('a', 'b')
`Map`	('key': value, 'k2': v2)
`Function`	get-function('name')

SASS:MAP FUNCTIONS

FUNCTION	RETURNS
`map.get($m, $k)`	Value at key (or null)
`map.set($m, $k, $v)`	New map with key set
`map.merge($a, $b)`	Shallow merge
`map.deep-merge($a, $b)`	Deep merge
`map.remove($m, $k)`	Map without key
`map.keys($m)`	List of all keys
`map.values($m)`	List of all values
`map.has-key($m, $k)`	Boolean

VARIABLE SCOPE

CONTEXT	SCOPE
`Top-level`	Module-level (not global to @use)
`Inside rule/mixin`	Local; doesn't leak out
`!global flag`	Writes to module-level from inside block
`!default flag`	Only sets if not yet defined
`@use 'file' with ($v: x)`	Override !default vars on import

⚠ COMMON MISTAKES

- Using Sass variables for runtime theming — Sass variables are compile-time; CSS custom properties are needed for JavaScript-toggled themes
- Forgetting 1-based indexing on lists — `list.nth($list, 1)` is the first item, not index 0
- Accessing map keys without quotes — `map.get($m, primary)` vs `map.get($m, 'primary')` — unquoted keys are treated as Sass values

✓ TIPS

- Use maps for design token systems — store color shades as a nested map for programmatic access
- Use `!default` in all library/shared files — lets consumers override tokens without editing source
- Generate utility classes from maps with `@each` — one map drives dozens of generated classes

SUMMARY

- `$var: value` — compile-time variable; lexically scoped
- `!default` — set only if not already defined (library pattern)
- `('key': value)` — Sass map; access with `map.get()`
- `@each $k, $v in $map` — iterate map to generate utility classes

Sass/SCSS — Nesting & BEM

& · &__ · &-- · @at-root · nesting rules · @media inside rule

NESTING & BEM PATTERNS

```scss
// Basic nesting
.nav {
  display: flex;
  gap: 1rem;

  a {                          // generates .nav a
    color: inherit;
    text-decoration: none;
  }

  &:hover { opacity: 0.8; }   // generates .nav:hover
}

// BEM with &
.card {
  padding: 1.5rem;
  border-radius: 8px;

  // Elements
  &__title {                   // generates .card__title
    font-size: 1.25rem;
    font-weight: 600;
  }
  &__body { color: #64748b; }
  &__footer { border-top: 1px solid #e2e8f0; }

  // Modifiers
  &--featured {                // generates .card--featured
    border: 2px solid #6366f1;
  }
  &--compact {
    padding: 0.75rem;
  }
}

// Responsive inside component
.hero {
  font-size: 3rem;

  @media (max-width: 640px) {   // generates @media inside .hero
    font-size: 2rem;
  }
}

// @at-root — escape nesting to top level
.theme-dark {
  .card {
    @at-root .card { background: #1e293b; }  // .card (not .theme-dark .card)
  }
}

// Deep selectors (avoid overuse)
.sidebar {
  > ul > li > a { color: #334155; }    // direct children chain
  ~ .content { margin-left: 1rem; }    // sibling combinator
}
```

NESTING EXPLAINED

- → `a {}` inside a rule — generates descendant selector
- → `&` — substituted with the full parent selector string
- → `&:hover` → `.card:hover`
- → `&__title` → `.card__title` (BEM element)
- → `&--featured` → `.card--featured` (BEM modifier)
- → `@media inside rule` — Sass bubbles it up to a top-level media query
- → `@at-root` — emits rule at stylesheet root, escaping parent nesting
- → `& .child` → `.parent .child` — child via space
- → `& + &` → `.item + .item` — adjacent sibling

PARENT SELECTOR PATTERNS

SASS	CSS OUTPUT
`.a { &:hover }`	.a:hover
`.a { &__b }`	.a__b
`.a { &--b }`	.a--b
`.a { .b & }`	.b .a (reversed)
`.a { &-b }`	.a-b
`.a { & + & }`	.a + .a
`.a { &::before }`	.a::before

BEM WITH SASS

BEM ROLE	PATTERN
Block	.card { }
Element	&__title { } → .card__title
Modifier	&--featured { } → .card--featured
Modifier on element	&__btn--active { }
State modifier	&.is-active { } → .card.is-active

NESTING BEST PRACTICES

RULE	REASON
Max 3 levels deep	Deeper = high specificity, hard to override
Use & for pseudo/modifier	Clear intent; doesn't add specificity
Nest @media inside rule	Keeps component styles together
Avoid tag selectors in nests	Creates fragile selector chains
BEM reduces need for nesting	Flat selectors with clear hierarchy

⚠ COMMON MISTAKES

- Nesting more than 3-4 levels — generates highly specific selectors that are nearly impossible to override
- Using `&__elem` inside another BEM element — generates `.block__elem1__elem2`, which is invalid BEM
- Nesting unrelated styles together — nesting is for related states/elements only; unrelated components should be separate rules

✓ TIPS

- Keep BEM element classes at the top level in Sass for searchability — don't hide `.card__title` three levels deep
- Use `.b &` {} (reversed parent) for context-dependent overrides like `.theme-dark .card`
- Use `@at-root` with BEM when you need a modifier at the block level but you're inside an element rule

SUMMARY

- `&` — parent selector; enables BEM, pseudo-classes, modifiers
- `&__elem / &--mod` — BEM elements and modifiers via concatenation
- `@media inside rule` — keeps responsive styles co-located with component
- `@at-root` — emit selector at stylesheet root, escaping nesting

Sheet 30

Sass/SCSS — Mixins & Functions

@mixin · @include · @function · @return · @content · @if · @each · @for

MIXINS & FUNCTIONS

```scss
// ── Mixins — output CSS declarations ──────────────────
@mixin flex-center {
  display: flex;
  align-items: center;
  justify-content: center;
}

@mixin truncate($width: 100%) {
  width: $width;
  overflow: hidden;
  white-space: nowrap;
  text-overflow: ellipsis;
}

@mixin respond-to($bp) {
  $breakpoints: ('sm': 640px, 'md': 768px, 'lg': 1024px);
  @media (min-width: map.get($breakpoints, $bp)) { @content; }
}

.card { @include flex-center; }
.title { @include truncate(200px); }

.hero {
  font-size: 3rem;
  @include respond-to('md') { font-size: 2rem; }
}

// ── Functions — return a value ────────────────────────
@use 'sass:math';

@function rem($px, $base: 16) {
  @return math.div($px, $base) * 1rem;
}

@function clamp-size($min, $max, $vw: 100vw) {
  @return clamp(#{$min}, calc(#{$vw} * 0.05), #{$max});
}

$h1: rem(48);    // 3rem
$body: rem(16);  // 1rem

// ── Control flow ──────────────────────────────────────
@mixin theme-colors($dark: false) {
  @if $dark {
    background: #0f172a; color: #f1f5f9;
  } @else {
    background: #ffffff; color: #0f172a;
  }
}

@for $i from 1 through 12 {
  .col-#{$i} { width: math.div(100%, 12) * $i; }
}

$sizes: (sm: 0.875rem, md: 1rem, lg: 1.25rem);
@each $name, $size in $sizes {
  .text-#{$name} { font-size: $size; }
}
```

EXPLAINED

- → `@mixin name($arg: default)` — define a reusable CSS block
- → `@include name(args)` — insert mixin output at call site
- → `@content` — slot for content block passed to mixin (like a yield)
- → `@function name` — returns a value; no CSS output
- → `@return value` — exits function and returns value
- → `@if / @else if / @else` — conditional logic
- → `@for $i from 1 through 12` — inclusive loop (1 to 12)
- → `@each $k, $v in $map` — map iteration
- → `1rem` multiplication — multiply a unitless number by `1rem` to get rem unit

MIXIN VS FUNCTION

FEATURE	@MIXIN	@FUNCTION
Output	CSS declarations	A value
Called with	@include	Function call
@content	Yes	No
Use in property	No	Yes
Example	@include flex-center	rem(16)

CONTROL FLOW

DIRECTIVE	USE CASE
`@if / @else if / @else`	Conditional output
`@for $i from a through b`	Inclusive range loop
`@for $i from a to b`	Exclusive range loop
`@each $item in $list`	Iterate list
`@each $k, $v in $map`	Iterate map pairs
`@while $condition`	Loop while true
`@error 'msg'`	Halt with error message
`@warn 'msg'`	Print warning, continue

COMMON MIXIN PATTERNS

MIXIN PURPOSE	KEY DECLARATIONS
Flex center	display:flex · align/justify: center
Absolute fill	position:absolute · inset:0
Visually hidden	sr-only accessible hide pattern
Truncate	overflow:hidden · text-overflow:ellipsis
Responsive	@media + @content slot
Focus ring	outline + offset + color
Clearfix	&::after { clear:both }

⚠ COMMON MISTAKES

- Using a function where a mixin is needed — functions return values; they can't output CSS declarations directly
- Forgetting `@return` in a function — without it the function returns null silently
- Trying to output CSS from `@function` — any CSS inside a function is invalid; use `@mixin` instead

✓ TIPS

- Use keyword arguments for clarity: `@include respond-to($bp: 'md')`
- Use `@content` in breakpoint mixins for cleaner component-level responsiveness
- Use `@error` in mixins for invalid argument checks — better than silent failures

SUMMARY

- `@mixin / @include` — reusable CSS declaration blocks
- `@function / @return` — compute and return a value
- `@content` — slot inside mixin for caller-provided CSS block
- `@each $k, $v in $map` — generate utility classes from token maps

Sass/SCSS — Module System

@use · @forward · @use with · namespace · private · index files · partials

@USE & @FORWARD

```scss
// ── File structure ──────────────────────────────────
// tokens/_colors.scss
$primary: #6366f1 !default;
$secondary: #64748b !default;

// tokens/_spacing.scss
$base: 1rem !default;
$sm: $base * 0.5;
$lg: $base * 1.5;

// tokens/_index.scss — barrel file
@forward 'colors';
@forward 'spacing';
@forward 'mixins';

// ── @use — consume a module ─────────────────────────
// Namespace access (default: filename)
@use 'tokens/colors';
.btn { background: colors.$primary; }

// Custom namespace
@use 'tokens/colors' as t;
.btn { background: t.$primary; }

// No namespace (use with caution)
@use 'tokens/colors' as *;
.btn { background: $primary; }

// ── @forward — re-export from barrel ────────────────
// Selective forward with prefix
@forward 'colors' with ($primary: #4f46e5 !default);
@forward 'mixins' show flex-center, truncate;  // only these
@forward 'utils' hide -private-fn;             // hide private

// ── Private members ─────────────────────────────────
$_internal-size: 14px;  // _ or - prefix = private (not @forward-able)

// ── @use with — override !default variables ─────────
@use 'bootstrap/scss/bootstrap' with (
  $primary: #6366f1,
  $body-font-family: 'Inter',
  $grid-gutter-width: 1.5rem
);
```

MODULE SYSTEM EXPLAINED

- → `@use 'path'` — imports a module; runs once per file regardless of how many times used
- → `colors.$primary` — namespace-qualified access to module member
- → `@use 'path' as ns` — custom namespace alias
- → `@use 'path' as *` — no namespace; names available directly (collision risk)
- → `@forward 'path'` — re-exports module members so they're accessible to consumers of this file
- → `@forward with ($var: val !default)` — configure !default variables on forward
- → `show / hide` — whitelist or blacklist members in a forward
- → `$_private` — underscore prefix makes member non-forwardable
- → `@use 'lib' with ($var: val)` — override !default vars at the @use site

@USE VS @IMPORT

FEATURE	@IMPORT	@USE
Namespace	None (global)	Filename by default
Runs per file	Every @import	Once per compilation
Variables scope	Global	Module-scoped
Private members	No	_ / - prefix
Status	Deprecated	Current standard

@FORWARD OPTIONS

SYNTAX	EFFECT
`@forward 'file'`	Re-export all members
`@forward 'file' show a, b`	Export only a and b
`@forward 'file' hide c`	Export all except c
`@forward 'file' as prefix-*`	Add prefix to all names
`@forward 'file' with ($v: x)`	Configure defaults

RECOMMENDED STRUCTURE

FILE / FOLDER	CONTENTS
`tokens/_colors.scss`	Color variables
`tokens/_spacing.scss`	Spacing scale
`tokens/_index.scss`	@forward all tokens
`mixins/_index.scss`	@forward all mixins
`components/_card.scss`	Card component styles
`main.scss`	@use everything, output CSS

⚠ COMMON MISTAKES

- › Trying to use `@use` and `@import` in the same file — they conflict; migrate fully to `@use` / `@forward`
- › Forgetting `@forward` in index files — `@use` alone does not re-export; barrel files need `@forward`
- › Expecting variables to be globally available after `@use` — each file must `@use` the module it needs; there's no global state

✓ TIPS

- › Create `_index.scss` barrel files in every folder — lets consumers `@use 'tokens'` instead of long paths
- › Use `@forward 'file' as prefix-*` to namespace tokens by category: `color-primary`, `space-md`
- › Audit any remaining `@import` usage — Dart Sass will remove support in a future version

SUMMARY

- › `@use 'path'` — import module; access via `filename.$var`
- › `@forward 'path'` — re-export module members from a barrel file
- › `@use 'lib' with ($var: val)` — configure !default variables at import
- › `$_private` — underscore prefix prevents member from being forwarded

PostCSS — Setup & Config

WHAT IS POSTCSS?

PostCSS is a tool for transforming CSS with JavaScript plugins. Unlike Sass, it is not a preprocessor — it processes valid (or near-valid) CSS through a plugin pipeline. Each plugin reads the CSS AST, transforms it, and passes it on. Common use cases include **autoprefixing**, **future CSS syntax** polyfills, linting, minification, and custom transformations. Tailwind CSS and CSS Modules both run through PostCSS.

KEY FACTS

- Install: `npm install -D postcss postcss-cli`
- Config file: `postcss.config.js` or `postcss.config.cjs`
- Vite, webpack (postcss-loader), and Parcel use PostCSS automatically
- Plugins run in array order — sequence matters
- PostCSS itself doesn't transform anything — plugins do all the work
- `autoprefixer` is the most-used plugin; used internally by many frameworks

SETUP & CONFIG

```
// Install
// npm install -D postcss postcss-cli autoprefixer postcss-preset-env cssnano

// postcss.config.js
module.exports = {
  plugins: [
    require('postcss-import'),           // resolve @import at build time
    require('postcss-preset-env')({      // future CSS → current CSS
      stage: 2,                          // 0=experimental 4=stable
      features: {
        'nesting-rules': true,           // enable CSS nesting polyfill
        'custom-media-queries': true,
        'color-function': true,
      },
    }),
    require('autoprefixer'),             // add vendor prefixes
    process.env.NODE_ENV === 'production'
      ? require('cssnano')({ preset: 'default' })
      : false,
  ],
}

// Vite — PostCSS config is auto-detected from postcss.config.js
// OR inline in vite.config.js:
import autoprefixer from 'autoprefixer'
export default {
  css: {
    postcss: {
      plugins: [autoprefixer()],
    },
  },
}

// CLI usage
// npx postcss src/main.css -o dist/main.css
// npx postcss src/main.css -o dist/main.css --watch
```

CONFIG EXPLAINED

- → `plugins` — ordered array of PostCSS plugin instances
- → `postcss-import` — inlines `@import` statements at build time
- → `postcss-preset-env` — polyfills future CSS based on browserslist
- → `stage` — CSS spec stage (0 experimental → 4 stable)
- → `autoprefixer` — adds `-webkit-`, `-moz-` prefixes from browserslist
- → `cssnano` — minifies CSS output; run only in production
- → `false` in plugins array — PostCSS skips falsy entries (conditional plugins)
- → Vite auto-detects `postcss.config.js` in project root

⚠ COMMON MISTAKES

- Wrong plugin order — `postcss-import` must be first so other plugins see the fully-inlined CSS
- Running `cssnano` in development — minification removes whitespace and makes debugging very hard
- Missing browserslist config — `autoprefixer` and `preset-env` need a target browser list in `package.json` or `.browserslistrc`

✓ TIPS

- Add `"browserslist": "> 0.5%, last 2 versions, not dead"` to `package.json` to configure autoprefixer and preset-env together
- Use `stage: 2` for preset-env — a good balance of stability and future features
- PostCSS plugins can be used alongside Sass — Sass compiles first, then PostCSS processes the output

SUMMARY

- `postcss.config.js` — array of plugins in processing order
- `autoprefixer` — vendor prefixes based on browserslist
- `postcss-preset-env` — future CSS syntax polyfills
- `cssnano` — minification; production only

PostCSS — Essential Plugins

autoprefixer · postcss-preset-env · cssnano · postcss-import · stylelint · purgecss

ESSENTIAL PLUGIN USAGE

```
// postcss.config.js — full production pipeline
module.exports = {
  plugins: [
    // 1. Resolve @import first
    require('postcss-import'),

    // 2. Future CSS features
    require('postcss-preset-env')({
      stage: 2,
      features: {
        'nesting-rules': true,
        'custom-media-queries': true,
        'media-query-ranges': true,   // @media (width >= 768px)
        'color-function': true,       // color(display-p3 …)
      },
    }),

    // 3. Vendor prefixes
    require('autoprefixer'),

    // 4. PurgeCSS — remove unused CSS
    require('@fullhuman/postcss-purgecss')({
      content: ['./src/**/*.{html,js,jsx,ts,tsx,vue}'],
      defaultExtractor: content => content.match(/[\w-/:]+(?<![:])/g) || [],
      safelist: ['active', 'is-visible', /^modal/],
    }),

    // 5. Minify (production only)
    process.env.NODE_ENV === 'production' && require('cssnano')({
      preset: ['default', { discardComments: { removeAll: true } }],
    }),
  ].filter(Boolean),
}

// Stylelint — CSS linting (separate config or postcss plugin)
// .stylelintrc.json
{
  "extends": "stylelint-config-standard",
  "rules": {
    "color-no-invalid-hex": true,
    "declaration-block-no-duplicate-properties": true,
    "no-duplicate-selectors": true
  }
}
```

PLUGIN NOTES

- → `postcss-import` — must run first; inlines all `@import` before other plugins process the CSS
- → `postcss-preset-env stage: 2` — polyfills W3C proposals at stage 2+ (candidate)
- → `nesting-rules: true` — polyfills CSS nesting syntax for older browsers
- → `media-query-ranges` — enables `@media (width >= 768px)` syntax
- → `autoprefixer` — reads browserslist, adds needed vendor prefixes
- → `purgecss content` — files to scan for used class names
- → `safelist` — class names/patterns never purged regardless of scan
- → `cssnano preset: 'default'` — safe minification (no breaking transforms)
- → `.filter(Boolean)` — removes `false` entries from conditional plugins

PLUGIN ECOSYSTEM

PLUGIN	PURPOSE
`autoprefixer`	Vendor prefixes from browserslist
`postcss-preset-env`	Future CSS polyfills
`cssnano`	Minification
`postcss-import`	Inline @import at build time
`postcss-purgecss`	Remove unused CSS selectors
`postcss-nested`	Sass-style nesting (lighter)
`postcss-custom-media`	@custom-media queries
`postcss-mixins`	CSS-style mixin syntax
`postcss-custom-properties`	CSS var() polyfill
`postcss-modules`	CSS Modules scoping

PRESET-ENV FEATURE FLAGS

FEATURE	ENABLES
`nesting-rules`	CSS nesting & → selector
`custom-media-queries`	@custom-media definitions
`media-query-ranges`	(width >= 768px) syntax
`color-function`	color() with display-p3
`color-mix`	color-mix() function
`has`	:has() selector polyfill
`is-pseudo-class`	:is() forgiving selector
`custom-properties`	CSS variables polyfill

BROWSERSLIST QUERIES

QUERY	MATCHES
`> 0.5%`	More than 0.5% global usage
`last 2 versions`	Last 2 versions each browser
`not dead`	Exclude abandoned browsers
`not IE 11`	Exclude IE 11
`supports es6-module`	Supports ES modules
`Chrome >= 110`	Chrome 110 and above
`defaults`	Standard recommended set

⚠ COMMON MISTAKES

- Running PurgeCSS without a safelist — dynamically added classes (from JS) get purged; always safelist them
- Using `postcss-nested` alongside `postcss-preset-env nesting-rules` — they conflict; use only one nesting plugin
- No browserslist configured — `autoprefixer` falls back to defaults; explicit targets give predictable output

✓ TIPS

- Use `npx browserslist` to see which browsers your current config targets
- Chain Sass → PostCSS: Sass compiles `.scss` first, then PostCSS processes the CSS output
- Use `cssnano preset: 'lite'` for safer, less aggressive minification that avoids edge-case bugs

SUMMARY

- Plugin order: import → preset-env → autoprefixer → purgecss → cssnano
- `autoprefixer` — reads browserslist to decide which prefixes to add
- `purgecss safelist` — protect dynamic and third-party classes from removal
- `.filter(Boolean)` — clean way to conditionally include plugins

PostCSS — Writing Plugins

postcss() · Declaration · Rule · AtRule · walk · replaceWith · plugin API

WRITING A POSTCSS PLUGIN

```
// my-plugin.js
const postcss = require('postcss')

// Plugin factory — returns an object with postcssPlugin + Once/Declaration/Rule
const myPlugin = postcss.plugin('my-plugin', (opts = {}) => {
  const { prefix = '' } = opts

  return (root, result) => {
    // Walk all declarations
    root.walkDecls('color', decl => {
      // decl.prop === 'color'
      // decl.value === '#fff'
      if (decl.value === 'red') {
        decl.value = 'var(--color-danger, #ef4444)'
      }
    })

    // Walk rules (selectors)
    root.walkRules(rule => {
      rule.selector = rule.selector.replace(
        /\.btn/g,
        `.${prefix}btn`
      )
    })

    // Walk @at-rules
    root.walkAtRules('custom-mixin', atRule => {
      const mixinName = atRule.params  // @custom-mixin flex-center → 'flex-center'
      atRule.replaceWith(
        postcss.decl({ prop: 'display', value: 'flex' }),
        postcss.decl({ prop: 'align-items', value: 'center' }),
        postcss.decl({ prop: 'justify-content', value: 'center' }),
      )
    })

    // Append a new rule to the stylesheet
    root.append(postcss.rule({
      selector: ':root',
      nodes: [postcss.decl({ prop: '--plugin-loaded', value: '1' })]
    }))
  }
})

// Modern API (PostCSS 8+)
module.exports = {
  postcssPlugin: 'my-plugin',
  Declaration: { color: decl => { /* handle color decls */ } },
  AtRule: { 'custom-mixin': atRule => { /* handle mixin */ } },
}
```

PLUGIN API EXPLAINED

- → `root` — the root CSS AST node (entire stylesheet)
- → `result` — PostCSS Result; use `result.warn()` for warnings
- → `root.walkDecls(prop, fn)` — visit every declaration; filter by property name
- → `root.walkRules(fn)` — visit every rule (selector block)
- → `root.walkAtRules(name, fn)` — visit specific @-rules
- → `decl.prop` — CSS property name string
- → `decl.value` — CSS value string; modify to transform
- → `atRule.replaceWith(...nodes)` — replace @-rule with new nodes
- → `postcss.decl()` — factory to create a Declaration node
- → `postcss.rule()` — factory to create a Rule node

CSS AST NODE TYPES

TYPE	EXAMPLE
`Root`	The whole stylesheet
`Rule`	.btn { } selector block
`Declaration`	color: red;
`AtRule`	@media, @keyframes, @custom
`Comment`	/* comment */

NODE METHODS

METHOD	ACTION
`node.remove()`	Remove from AST
`node.replaceWith(…)`	Replace with new nodes
`node.cloneAfter()`	Clone and insert after
`node.cloneBefore()`	Clone and insert before
`parent.append(node)`	Add node at end
`parent.prepend(node)`	Add node at start
`result.warn('msg')`	Emit a warning
`node.error('msg')`	Throw with source position

WALK METHODS

METHOD	VISITS
`root.walk(fn)`	All nodes
`root.walkDecls(fn)`	All declarations
`root.walkDecls(prop, fn)`	Decls by property
`root.walkRules(fn)`	All selector rules
`root.walkAtRules(fn)`	All @-rules
`root.walkAtRules(name, fn)`	@-rule by name
`root.walkComments(fn)`	All comments

⚠ COMMON MISTAKES

- Mutating `decl.prop` while walking — some walkers use the original prop for filtering; modify after the condition check
- Not returning from plugin factory — the factory must return a function (or for PostCSS 8, the object with handlers)
- Forgetting async support — if your plugin does async work, return a Promise or use async/await in the handler

✓ TIPS

- Use `postcss-plugin-boilerplate` or the official PostCSS plugin template to get testing set up
- Use `node.source.start.line` and `.column` for error messages with accurate source positions
- Test plugins with `postcss([myPlugin]).process(css).css` — simple unit testing without a bundler

SUMMARY

- `root.walkDecls(prop, fn)` — visit and transform CSS declarations
- `root.walkAtRules(name, fn)` — intercept and replace custom @-rules
- `decl.replaceWith()` / `atRule.replaceWith()` — swap nodes in the AST
- `postcss.decl()` / `postcss.rule()` — factory methods to create new nodes

Open Props — Setup & Import

WHAT IS OPEN PROPS?

Open Props is a collection of CSS custom properties (design tokens) that cover the most common design decisions — colors, spacing, typography, shadows, gradients, borders, animations, and more. It is **framework-agnostic** and works anywhere CSS runs. Rather than inventing tokens, you adopt a well-considered system with excellent defaults. The `postcss-jit-props` plugin ensures only the custom properties you actually use are included in the build output.

SETUP OPTIONS

- **CDN:** `unpkg.com/open-props` — easiest for prototyping
- **npm:** `npm install open-props`
- **JIT:** `postcss-jit-props` — ships only used props
- Props file: `open-props/props.all.css` — ~40KB, all props
- Normalize: `open-props/normalize.css` — sensible element defaults using Open Props tokens
- Author: Adam Argyle (Chrome team)

IMPORT STRATEGIES

```
/* ── CDN (all props) ──────────────────────────────── */
@import "https://unpkg.com/open-props";
@import "https://unpkg.com/open-props/normalize.css";

/* ── npm (all props) ──────────────────────────────── */
@import "open-props";
@import "open-props/normalize";

/* ── Selective import (smaller) ──────────────────── */
@import "open-props/colors";
@import "open-props/sizes";
@import "open-props/fonts";
@import "open-props/shadows";
@import "open-props/animations";

/* ── PostCSS JIT (best for production) ──────────── */
/* postcss.config.js:
   require('postcss-jit-props')(OpenProps)
   — only the custom props you USE are emitted */
```

```
/* Using Open Props in your CSS */
.card {
  background: var(--surface-1);
  border: var(--border-size-1) solid var(--surface-3);
  border-radius: var(--radius-2);
  padding: var(--size-4);
  box-shadow: var(--shadow-2);
}

.card__title {
  font-size: var(--font-size-3);
  font-weight: var(--font-weight-6);
  color: var(--text-1);
}

.btn-primary {
  background: var(--indigo-6);
  color: var(--indigo-0);
  padding: var(--size-2) var(--size-4);
  border-radius: var(--radius-full);
}
```

TOKEN SYSTEM EXPLAINED

- → `--size-{1-15}` — spacing scale from 0.25rem to 15rem
- → `--font-size-{0-8}` — type scale from 0.75rem to 3.5rem
- → `--color-{0-15}` — per-palette shade 0 (lightest) to 15 (darkest)
- → `--surface-{1-4}` — neutral surface colors (adapt in dark mode)
- → `--text-{1-4}` — text colors (1=primary, 2=secondary)
- → `--shadow-{1-6}` — elevation shadows small to large
- → `--radius-{1-6}` — border-radius from 2px to 24px
- → `--border-size-{1-5}` — border widths 1px to 5px
- → `postcss-jit-props` — tree-shakes unused custom properties

⚠ COMMON MISTAKES

- Importing all of Open Props without JIT in production — 40KB of custom properties you mostly don't use; always use JIT or selective imports
- Expecting colors to adapt to dark mode automatically — only `--surface-*` and `--text-*` adapt; palette tokens like `--indigo-6` do not change
- Using wrong prop names — props are `--size-4` not `--spacing-4`; check the docs for exact names

✓ TIPS

- Use `open-props/normalize` — it provides a beautiful typographic baseline using Open Props tokens
- Open Props dark mode adapts `--surface-*` and `--text-*` tokens automatically via `@media (prefers-color-scheme: dark)`
- Browse all tokens at `open-props.style` — interactive token explorer with live previews

SUMMARY

- `@import "open-props"` — load all design tokens as CSS custom properties
- `postcss-jit-props` — emit only the props you actually use
- `--surface-* / --text-*` — adaptive tokens that respond to dark mode
- `open-props/normalize` — element baseline styles built on Open Props tokens

Open Props — Token Reference

--size · --font-size · --color · --shadow · --radius · --font-weight · --ease

SIZE SCALE (--SIZE-*)

TOKEN	VALUE
--size-1	0.25rem (4px)
--size-2	0.5rem (8px)
--size-3	0.75rem (12px)
--size-4	1rem (16px)
--size-5	1.25rem (20px)
--size-6	1.5rem (24px)
--size-7	1.75rem (28px)
--size-8	2rem (32px)
--size-10	2.5rem (40px)
--size-12	3rem (48px)
--size-14	3.5rem (56px)
--size-15	4rem (64px)

TYPOGRAPHY TOKENS

TOKEN	VALUE
--font-size-0	0.75rem (12px)
--font-size-1	1rem (16px)
--font-size-2	1.1rem (17.6px)
--font-size-3	1.25rem (20px)
--font-size-4	1.5rem (24px)
--font-size-5	2rem (32px)
--font-size-6	2.5rem (40px)
--font-size-7	3rem (48px)
--font-size-8	3.5rem (56px)
--font-weight-1 … 9	100 … 900
--font-lineheight-0	1.1 (tight)
--font-lineheight-3	1.75 (relaxed)

SHADOWS, RADIUS, BORDERS

TOKEN	VALUE / EFFECT
--shadow-1	Subtle elevation
--shadow-2	Card elevation
--shadow-3	Dropdown elevation
--shadow-4	Modal elevation
--shadow-5	Large overlay
--shadow-6	Dramatic shadow
--radius-1	2px
--radius-2	5px
--radius-3	1rem
--radius-full	1e5px (pill)
--border-size-1	1px
--border-size-3	3px

COLOR PALETTES

PALETTE	SHADES
--gray-{0-15}	Neutral grays
--red-{0-15}	Red scale
--orange-{0-15}	Orange scale
--yellow-{0-15}	Yellow scale
--green-{0-15}	Green scale
--teal-{0-15}	Teal scale
--blue-{0-15}	Blue scale
--indigo-{0-15}	Indigo scale
--violet-{0-15}	Violet scale
--pink-{0-15}	Pink scale
--surface-{1-4}	Adaptive surfaces
--text-{1-4}	Adaptive text

EASING & ANIMATION

TOKEN	CURVE
--ease-1	Mild deceleration
--ease-2	Standard ease-out
--ease-3	Expressive ease-out
--ease-4	Strong ease-out
--ease-in-1 … 4	Ease-in variants
--ease-elastic-1 … 4	Spring-like overshoot
--ease-squish-1 … 5	Squish bounce easing
--animation-fade-in	Fade-in keyframe ref
--animation-slide-in-up	Slide up keyframe ref

GRADIENTS & OTHERS

TOKEN	EFFECT
--gradient-1 … 30	Pre-built gradients
--noise-1 … 5	Noise texture overlays
--layer-1 … 5	z-index scale 1–5
--layer-important	z-index: 2147483647
--size-fluid-1 … 10	Fluid (clamp) sizes
--size-content-1 … 3	Prose content widths
--size-header-1 … 3	Header-optimized widths

⚠ COMMON MISTAKES

- Treating palette tokens as semantic — `--indigo-6` is not adaptive; create semantic tokens like `--color-accent: var(--indigo-6)`
- Confusing `--size-*` (spacing) with `--font-size-*` (type) — both start at 1 but have different value ranges
- Using gradient tokens in production without JIT — all 30 gradients are large SVG data URIs; JIT ensures only used ones are included

✓ TIPS

- Build a semantic token layer on top: `:root { --color-brand: var(--indigo-6); }` — insulates your code from Open Props version changes
- Use `--ease-elastic-*` for button press animations — they create satisfying spring-like feedback
- Use `--size-fluid-*` for heading font sizes — clamp-based so they resize fluidly without media queries

SUMMARY

- `--size-{1-15}` — spacing scale; 4 = 1rem
- `--font-size-{0-8}` — type scale; 1 = 1rem
- `--shadow-{1-6}` — elevation shadows; 2 = card elevation
- `--surface-* / --text-*` — adaptive tokens that respond to dark mode

Open Props — Adaptive & Dark Mode

prefers-color-scheme · --surface-* · postcss-jit-props · semantic tokens · dark:*

ADAPTIVE THEMING

```
/* 1. Import Open Props */
@import "open-props";
@import "open-props/normalize";

/* 2. Build semantic token layer */
:root {
  --color-bg:       var(--gray-0);
  --color-surface:  var(--gray-1);
  --color-text:     var(--gray-9);
  --color-muted:    var(--gray-6);
  --color-border:   var(--gray-3);
  --color-accent:   var(--indigo-6);
  --color-accent-2: var(--indigo-1);
}

/* 3. Dark mode — override semantic layer only */
@media (prefers-color-scheme: dark) {
  :root {
    --color-bg:       var(--gray-9);
    --color-surface:  var(--gray-8);
    --color-text:     var(--gray-1);
    --color-muted:    var(--gray-5);
    --color-border:   var(--gray-7);
    --color-accent:   var(--indigo-4);
    --color-accent-2: var(--indigo-9);
  }
}

/* 4. Class-based dark mode (JS toggle) */
[data-theme="dark"] {
  --color-bg:       var(--gray-9);
  --color-surface:  var(--gray-8);
  --color-text:     var(--gray-1);
}

/* 5. Use semantic tokens everywhere */
body { background: var(--color-bg); color: var(--color-text); }

.card {
  background: var(--color-surface);
  border: 1px solid var(--color-border);
  border-radius: var(--radius-2);
  padding: var(--size-4);
}

/* 6. Open Props built-in adaptive tokens */
body {
  background: var(--surface-1); /* Auto-adapts to dark mode */
  color: var(--text-1);         /* Auto-adapts */
}

/* 7. High-contrast support */
@media (prefers-contrast: more) {
  :root {
    --color-border: var(--gray-9);
    --color-text:   var(--gray-9);
  }
}
```

ADAPTIVE PATTERN

- → `--surface-* / --text-*` — Open Props built-in adaptive tokens; auto-flip in dark mode
- → Custom semantic layer isolates your code from specific palette values
- → `prefers-color-scheme: dark` — system dark mode preference
- → `:root` overrides only the semantic tokens — palette tokens remain unchanged
- → `[data-theme="dark"]` — class/attr toggle for JS-controlled dark mode
- → `prefers-contrast: more` — increases contrast for accessibility
- → Never reference palette tokens (`--indigo-6`) directly in components — go through semantic layer

BUILT-IN ADAPTIVE TOKENS

TOKEN	LIGHT MODE	DARK MODE
`--surface-1`	--gray-0	--gray-9
`--surface-2`	--gray-1	--gray-8
`--surface-3`	--gray-2	--gray-7
`--surface-4`	--gray-3	--gray-6
`--text-1`	--gray-9	--gray-1
`--text-2`	--gray-7	--gray-3

SEMANTIC TOKEN NAMING

CATEGORY	EXAMPLE NAMES
Backgrounds	--color-bg · --color-surface · --color-overlay
Text	--color-text · --color-muted · --color-heading
Borders	--color-border · --color-divider
Interactive	--color-accent · --color-focus · --color-hover
Status	--color-success · --color-warning · --color-danger

POSTCSS JIT SETUP

STEP	CONFIG
Install	`npm i -D postcss-jit-props`
Import OpenProps	`const OpenProps = require('open-props')`
Add plugin	`require('postcss-jit-props')(OpenProps)`
Remove @import	JIT injects used props automatically
Result	Only referenced props in output

⚠ COMMON MISTAKES

- Using both `@import "open-props"` and JIT plugin together — JIT replaces the import; using both injects props twice
- Directly using palette tokens in components (`var(--indigo-6)`) — when you later need dark mode, you must find and update every usage
- Forgetting to test with OS dark mode enabled — browser dark mode preview in DevTools doesn't always match real OS setting

✓ TIPS

- Test adaptive themes with `prefers-color-scheme: dark` in Chrome DevTools Rendering panel
- Support both media query and class toggle — some users want manual override independent of OS setting
- Use CSS `color-scheme: light dark` on `:root` — lets the browser adapt native UI controls (scrollbars, inputs) too

SUMMARY

- Semantic token layer — maps palette tokens to purpose; override only the layer for dark mode
- `--surface-* / --text-*` — Open Props tokens that adapt automatically
- `prefers-color-scheme: dark` — OS-level dark mode query
- `[data-theme="dark"]` — JS-toggleable dark mode override

ITCSS — Architecture & Layers

settings · tools · generic · elements · objects · components · utilities · specificity triangle

WHAT IS ITCSS?

ITCSS (Inverted Triangle CSS) is a CSS architecture methodology by Harry Roberts. It organizes CSS into layers ordered by increasing specificity, from **widest reach / lowest specificity** (settings, tools) to **narrowest reach / highest specificity** (utilities). This prevents specificity conflicts and makes overrides predictable. ITCSS is a methodology, not a framework — it works with any preprocessor or plain CSS.

CORE PRINCIPLES

- Specificity only ever increases down the triangle
- Layer order = import order in your main CSS file
- No selector is ever undone by a later selector at the same layer
- Works with BEM, SMACSS, or any naming convention
- Pairs naturally with Sass `@use` / `@forward` for file organization
- Utilities layer (highest specificity) is intentionally last

THE SEVEN LAYERS

#	LAYER	CONTENTS	SPECIFICITY
1	`Settings`	Variables, tokens, config	None
2	`Tools`	Mixins, functions	None
3	`Generic`	Resets, normalize, box-sizing	Element
4	`Elements`	Bare HTML element styles	Element
5	`Objects`	Layout patterns (media, pack, etc.)	Class
6	`Components`	UI components (card, nav, btn)	Class
7	`Utilities`	Helper overrides (hidden, truncate)	Class + !important

FILE STRUCTURE

FOLDER	FILES
`1-settings/`	_colors.scss · _spacing.scss · _typography.scss
`2-tools/`	_mixins.scss · _functions.scss
`3-generic/`	_reset.scss · _box-sizing.scss
`4-elements/`	_headings.scss · _links.scss · _images.scss
`5-objects/`	_media.scss · _flag.scss · _pack.scss
`6-components/`	_card.scss · _nav.scss · _button.scss
`7-utilities/`	_visibility.scss · _spacing.scss · _text.scss
`main.scss`	@use each layer in order

ITCSS VS OTHER METHODOLOGIES

METHODOLOGY	FOCUS
`ITCSS`	Layer ordering by specificity
`BEM`	Naming convention (Block__Elemen -Modifier)
`SMACSS`	Category-based file organization
`OOCSS`	Object-oriented reusable pattern
`Atomic CSS`	Single-purpose utility classes
`CSS Layers`	Native @layer cascade orderin

⚠ COMMON MISTAKES

- Putting component-specific overrides in the Objects layer — objects should be abstract layout patterns, not visual components
- Writing high-specificity selectors in early layers — Settings and Tools produce no CSS; Generic and Elements should use only element selectors
- Skipping layers — even if a layer has few rules, maintain the structure so future additions have a clear home

✓ TIPS

- Combine ITCSS layers with CSS `@layer` — native cascade layers enforce the ordering at the browser level
- The Objects layer is for layout patterns only — no colors, no typography; those live in Components
- Utilities should use `!important` — they represent intentional, final overrides that shouldn't be defeated by anything

SUMMARY

- 7 layers: settings → tools → generic → elements → objects → components → utilities
- Specificity only increases downward — no layer undoes a previous layer
- Objects = abstract layout; Components = visual UI; Utilities = override helpers
- Import order in main.scss enforces the specificity triangle

ITCSS — Implementation Patterns

ITCSS IN PRACTICE

```scss
// ── main.scss — import order is the layer order ──────────

// 1. Settings — variables only, no CSS output
@use '1-settings/colors';
@use '1-settings/spacing';
@use '1-settings/typography';

// 2. Tools — mixins/functions, no CSS output
@use '2-tools/mixins';
@use '2-tools/functions';

// 3. Generic — global resets
@use '3-generic/reset';
@use '3-generic/box-sizing';

// 4. Elements — bare HTML elements
@use '4-elements/headings';
@use '4-elements/links';
@use '4-elements/images';

// 5. Objects — abstract layout patterns (no visual styling)
@use '5-objects/media';        // .o-media { display:flex }
@use '5-objects/container';    // .o-container { max-width: }
@use '5-objects/grid';         // .o-grid { display:grid }

// 6. Components — designed UI pieces
@use '6-components/button';
@use '6-components/card';
@use '6-components/navbar';

// 7. Utilities — single-purpose overrides (highest specificity)
@use '7-utilities/visibility'; // .u-hidden { display:none !important }
@use '7-utilities/spacing';    // .u-mt-4 { margin-top: 1rem !important }
@use '7-utilities/text';       // .u-truncate { overflow:hidden ... }

// ── With native CSS @layer ──────────────────────────────
@layer settings, tools, generic, elements, objects, components, utilities;
// @layer order declared upfront; later declarations follow this order
```

LAYER RULES

- → `settings` — Sass variables; zero CSS output; used by everything else
- → `tools` — mixins and functions; zero CSS output
- → `generic` — element selectors only; lowest specificity CSS
- → `elements` — styled bare HTML tags (`h1` , `a` , `img`)
- → `objects` — prefixed `.o-` ; layout only, no visual styles
- → `components` — prefixed `.c-` ; visual UI pieces
- → `utilities` — prefixed `.u-` ; use `!important` intentionally
- → `@layer` declaration — upfront order declaration for native cascade layers

CLASS NAMING PREFIXES

PREFIX	LAYER	EXAMPLE
`o-`	Objects	o-media o-grid
`c-`	Components	c-card c-btn
`u-`	Utilities	u-hidden u-mt-4
`is- / has-`	State modifiers	is-active has-error
`js-`	JS hooks (no styling)	js-toggle js-modal
`t-`	Theme variations	t-dark t-brand

COMMON OBJECT PATTERNS

OBJECT	PATTERN
`o-container`	Max-width centered wrapper
`o-media`	Flex: image + text content
`o-flag`	Media object with vertical align
`o-grid`	CSS Grid layout shell
`o-pack`	Flex children pack to edges
`o-stack`	Vertical stack with gap
`o-cluster`	Wrapping inline-flex group

NATIVE @LAYER MAPPING

@LAYER NAME	ITCSS LAYER
`reset`	Generic
`base`	Elements
`objects`	Objects
`components`	Components
`utilities`	Utilities
`(unlayered)`	Wins over all layers

⚠ COMMON MISTAKES

- Adding visual styles (colors, fonts) to Object layer classes — objects are layout only; add a Component skin on top
- Not declaring `@layer` order upfront — without the declaration, layers are ordered by first encounter which can be unpredictable
- Using utility classes in HTML and then overriding them in components — utilities are terminal; if you need to override, move the style to a component

✓ TIPS

- Use native CSS `@layer` for the Objects/Components/Utilities split — it enforces the ordering at browser level, not just file order
- Combine ITCSS with BEM — BEM names components within each layer; ITCSS controls where they live in the cascade
- Keep the `js-` prefix strictly for JavaScript hooks — never style these classes

SUMMARY

- `o- / c- / u-` — namespace prefixes for Objects, Components, Utilities
- Import order in main.scss defines cascade precedence
- Objects: layout only; Components: visual UI; Utilities: overrides
- Native `@layer` can enforce ITCSS ordering at the browser level

ITCSS — Scaling & Team Workflow

ITCSS AT SCALE

At scale, ITCSS needs additional conventions to remain maintainable. Key concerns are: **design token management** (keeping settings in sync with design tools), **team conventions** (naming, where new things go), **linting** (enforcing layer rules automatically), and **documentation** (keeping a living style guide). These practices prevent the architecture from degrading over time as teams grow and codebases expand.

SCALING CONCERNS

- Design tokens should live in settings and never be duplicated in components
- Use Stylelint to enforce import order and naming conventions
- Document each object and component with usage examples
- Define a decision tree: "Where does this new CSS go?"
- Use CSS `@layer` to make layer boundaries explicit and browser-enforced
- Design token pipeline: Figma tokens → Style Dictionary → settings layer

PLACEMENT DECISION TREE

QUESTION
Is it a variable
Is it a mixin or
Is it a global re
Is it a bare HTMI
Is it a layout pa
Is it a visual UI
Is it a single-pu

STYLELINT RULES FOR ITCSS

RULE	ENFORCES
`no-duplicate-selectors`	No repeated selector blocks
`selector-class-pattern`	Naming convention (o-/c-/u-)
`max-nesting-depth`	Limit selector specificity creep
`no-descending-specificity`	Higher-specificity before lower
`declaration-no-important`	Allow only in utilities layer
`import-notation`	Enforce @use over @import

DESIGN TOKEN PIPELINE

STAGE	TOOL
`Design source`	Figma / Tokens Studio
`Token format`	Design Tokens Community Group JSON
`Transform`	Style Dictionary / Token Transformer
`Output`	Sass variables / CSS custom properties
`ITCSS layer`	Settings (variables)
`Sync trigger`	CI/CD on token export

⚠ COMMON MISTAKES

- Hardcoding values in Components that duplicate Settings — creates drift between design tokens and actual usage
- Allowing ad-hoc utility classes to be added to components layer — utilities must stay in their own layer or specificity order breaks
- Not documenting the architecture — new team members default to wherever seems convenient without a decision tree or ADR

✓ TIPS

- Write an Architecture Decision Record (ADR) for CSS layer rules — formalize what goes where so decisions are documented
- Use a living style guide (Storybook, Fractal) tied to the components layer — keeps documentation and implementation in sync
- Add a CI step that lints CSS naming conventions — enforces `o-/c-/u-` prefixes automatically

SUMMARY

- Decision tree — tells every team member where new CSS belongs
- Stylelint rules — automate naming and specificity conventions
- Token pipeline — Figma → Style Dictionary → Settings layer
- ADR + living style guide — prevent architectural decay as team grows

Utility Class Comparison

TAILWIND VS UNOCSS

Both Tailwind CSS and UnoCSS provide utility-first class-based styling. **Tailwind** is the market leader with a large ecosystem, opinionated conventions, and the best tooling support. **UnoCSS** is an atomic CSS engine — more flexible and faster, with a preset system that can emulate Tailwind. The right choice depends on ecosystem needs, customization depth, and whether you want an opinionated framework or a configurable engine.

QUICK VERDICT

- **Choose Tailwind** for teams, large ecosystems, React/Next.js projects
- **Choose UnoCSS** for Vite-first, fine-grained control, custom utility engines
- UnoCSS with presetUno gives ~95% Tailwind compatibility
- Both support arbitrary values, dark mode, and responsive variants
- Tailwind v4 moves to CSS-first config (closer to UnoCSS model)

CORE COMPARISON

FEATURE	TAILWIND V3	UNOCSS
Type	Framework (opinionated)	Engine (flexible)
Config language	JS (tailwind.config.js)	TS (uno.config.ts)
Built-in utilities	Yes, large set	Via presets
JIT generation	Yes (default in v3)	Yes (always)
Arbitrary values	w-[342px]	w-[342px]
Attributify mode	No	Yes (presetAttributify)
Icons	Via plugin	Built-in (presetIcons)
Custom rules	Via plugins	rules array (simpler)
Build speed	Fast	Faster
Ecosystem	Very large	Growing

CLASS SYNTAX CROSSWALK

CONCEPT	TAILWIND	UNOCSS
Flex center	flex items-center justify-center	flex item center ju center
Hover color	hover:bg-blue-500	hover:bg blue-50
Responsive	md:grid-cols-2	md:grid- 2
Dark mode	dark:bg-gray-900	dark:bg- 900
Arbitrary	w-[342px]	w-[342p
Group hover	group group-hover:opacity-100	group gr hover:op 100

CONFIG COMPARISON

CONCEPT	TA
Extend theme	th
Custom class	@ {}
Alias	@ { .
Safelist	sa
Content scan	co
Dark mode	da

⚠ COMMON MISTAKES

- Assuming UnoCSS presetUno is 100% Tailwind-compatible — about 5% of advanced Tailwind features may need adjustment
- Using Tailwind CDN in production — it bundles every class; always use the PostCSS build pipeline for production
- Forgetting `import 'virtual:uno.css'` in the entry file — UnoCSS won't inject styles without this import

✓ TIPS

- Use UnoCSS's interactive inspector (`/__unocss`) during development to see what classes are being generated
- Both support the same Prettier plugin for class sorting — use `prettier-plugin-tailwindcss` with UnoCSS's Tailwind preset too
- Tailwind v4 (CSS-first config) will narrow the gap between the two approaches significantly

SUMMARY

- Tailwind — opinionated framework; large ecosystem; best tooling support
- UnoCSS — flexible engine; faster; preset-based; attributify mode
- Class syntax ~95% compatible when using presetUno
- Both: JIT, arbitrary values, dark mode, responsive variants

Component Framework Comparison

Bootstrap vs Bulma vs Foundation · grid · JS · Sass · file size · DX

BOOTSTRAP VS BULMA VS FOUNDATION

All three are component-first CSS frameworks. **Bootstrap** is the most popular with the largest ecosystem and Vanilla JS components. **Bulma** is CSS-only, zero JavaScript, with a clean readable class system. **Foundation** is enterprise-grade with an advanced XY grid and Motion UI animations. Your choice depends on whether you need built-in JS components, your grid requirements, and how much you customize via Sass.

QUICK VERDICT

- **Bootstrap** — broadest ecosystem, ready components, fastest start
- **Bulma** — CSS only, clean classes, great if you control your own JS
- **Foundation** — advanced grid, sticky + off-canvas, best for complex layouts
- All three fully customizable via Sass; all mobile-first

CORE DIFFERENCES

FEATURE	BOOTSTRAP 5
JS Components	Yes (Vanilla JS)
Grid	12-col Flexbox
Breakpoints	6 (xs–xxl)
Customization	Sass variables
CDN size (min+gz)	~22KB CSS
Naming	kebab .btn

GRID SYSTEM

CONCEPT	BOOTSTRAP	BULMA	FOUNDATION
Container	.container	.container	.grid-container
Row	.row	.columns	.grid-x
Column	.col-*	.column	.cell
12 col size	.col-md-8	.is-8	small-8 medium-8
Auto width	.col	.column (auto)	auto
Offset	.offset-md-3	.is-offset-3	medium-offset-3
Gutter	.g-4 (gap)	default / .is-gapless	grid-margin-x
Wrap extra	.row (new)	is-multiline	natural wrap

COMPONENT SUPPORT

COMPONENT	BOOTSTRAP	BU
Modal	Yes (JS)	CS JS
Dropdown	Yes (JS)	CS JS
Navbar responsive	Yes (JS toggle)	CS JS
Accordion	Yes	No
Toast	Yes	No (nc
Tooltip	Yes (JS init)	No
Sticky	Via utility	No
Off-canvas	Offcanvas	No
Form validation	Yes (classes)	No

⚠ COMMON MISTAKES

- Expecting Bulma to have built-in JS — it has none; all interactivity requires custom JavaScript
- Using Foundation without jQuery — Foundation 6 requires jQuery for its JS components
- Expecting identical grid column counts — Bootstrap uses 12; Bulma uses 12 numerically or fractions

✓ TIPS

- Bootstrap: use data-bs-* attributes for JS components — no JavaScript coding needed for most interactions
- Bulma: pair with Alpine.js or Stimulus for lightweight reactivity without the overhead of a full framework
- Foundation: use the XY grid for complex responsive layouts — it handles vertical grids that Bootstrap/Bulma can't

SUMMARY

- Bootstrap — largest ecosystem; Vanilla JS components; 12-col flex grid
- Bulma — CSS only; is-* modifiers; clean flexbox columns system
- Foundation — XY grid; advanced JS components; deepest Sass customization
- All three: mobile-first, Sass customizable, CDN available

CSS-in-JS Comparison

CSS-IN-JS ECOSYSTEM

CSS-in-JS approaches range from **runtime solutions** (styled-components, Emotion) that inject styles dynamically to **zero-runtime solutions** (Vanilla Extract, Linaria) that extract CSS at build time. **CSS Modules** occupy a middle ground — scoped CSS at build time with no runtime overhead. The right choice depends on your performance requirements, SSR needs, and how tightly you want styles coupled to component logic.

QUICK VERDICT

- **Styled Components / Emotion** — best DX; runtime cost; great for component libraries
- **Vanilla Extract / Linaria** — zero runtime; TypeScript-first; build-time extraction
- **CSS Modules** — zero runtime; no JS coupling; works with any CSS syntax
- RSC (React Server Components) favors zero-runtime or CSS Modules

CORE COMPARISON

FEATURE	STYLED-COMPOI
Runtime	Yes
TypeScript	Good
SSR support	ServerSt
Syntax	Tagged t
Theming	ThemePr
Props/dynamic	Yes (interpol
Bundle impact	~15KB

WHEN TO CHOOSE

USE CASE	BEST CHOICE
Design system / component lib	Styled-components or Emotion
SSR performance critical	Vanilla Extract or CSS Modules
Type-safe tokens + zero runtime	Vanilla Extract
React Server Components	CSS Modules or Vanilla Extract
Migrating from CSS	CSS Modules (minimal change)
Existing Tailwind project	Stay on Tailwind + CSS Modules
Vue / Svelte	CSS Modules or Panda CSS

EMOTION VS STYLED-COMPONENTS

FEATURE	STYLED-COMPONENTS
API	styled.div
Bundle size	~15KB
Performance	Slightly slower
MUI uses	No
css prop	No (by default)
Object styles	Yes
Community	Larger

⚠ COMMON MISTAKES

- Choosing runtime CSS-in-JS for SSR without ServerStyleSheet/streaming setup — causes flash of unstyled content
- Vanilla Extract: trying to use runtime values (component props) directly — VE is build-time only; use CSS custom properties for dynamic values
- CSS Modules: importing without the .module.css extension — regular .css files are global; the .module suffix is required for scoping

✓ TIPS

- For new React projects leaning toward RSC, prefer CSS Modules or Vanilla Extract — they have no runtime overhead
- Emotion offers a smaller bundle and marginally better performance than styled-components — meaningful at scale
- Panda CSS and StyleX (Meta) are emerging zero-runtime alternatives worth evaluating for large projects

SUMMARY

- Runtime (SC/Emotion) — best DX, dynamic props, theming; pay runtime cost
- Zero-runtime (VE/Linaria) — build-time extraction; TypeScript types; no runtime
- CSS Modules — simplest migration; scoped classes; works with any CSS
- Choose based on: SSR needs, TypeScript requirements, and bundle size budget

Preprocessor Comparison

Sass vs PostCSS vs Less · variables · nesting · mixins · @layer · compile speed

SASS VS POSTCSS VS LESS

Sass is the most feature-rich preprocessor with variables, mixins, functions, modules, and a mature ecosystem. **PostCSS** is a plugin-based CSS transformer — not a preprocessor, but an extensible pipeline that can do everything a preprocessor does and more. **Less** is a lighter preprocessor with a JavaScript-friendly syntax; largely overtaken by Sass. Modern projects increasingly use PostCSS alongside Sass rather than choosing one.

QUICK VERDICT

- **Sass/SCSS** — best for design token systems, component libraries, deep customization
- **PostCSS** — best for autoprefixing, future CSS, custom transforms, and pipeline composition
- **Less** — legacy; use only if maintaining an existing Less codebase
- **Combo:** Sass + PostCSS is very common — Sass for authoring, PostCSS for output transformation

FEATURE COMPARISON

FEATURE	SASS	POSTCSS	LESS
Variables	$var (compile-time)	CSS vars via plugin	@var (compile-time)
Nesting	Yes	postcss-nested / preset-env	Yes
Mixins	@mixin / @include	postcss-mixins	#.mixin()
Functions	@function	Custom plugin	Mixins with args
Modules	@use / @forward	postcss-import	@import (global)
Loops	@for @each @while	Manual plugin	Loops plugin
Type system	Rich (map, list, color)	None (plain CSS)	Limited
Future CSS	No	postcss-preset-env	No
Autoprefixer	No	Yes (plugin)	No
Compile speed	Fast (Dart Sass)	Depends on plugins	Fast

WHEN TO USE EACH

USE CASE	BEST TOOL
Design token system	Sass (maps, variables)
Component library	Sass (mixins, extend)
Vendor prefixes	PostCSS (autoprefixer)
Future CSS syntax	PostCSS (preset-env)
Custom transforms	PostCSS (write plugin)
Minification	PostCSS (cssnano)
CSS linting	PostCSS (stylelint)
Bootstrap / Foundation customization	Sass (required for config)
Tailwind customization	PostCSS (Tailwind uses it)
Existing Less codebase	Less (maintain as-is)

COMMON PIPELINE COMBOS

STACK
Tailwind o
Sass + Pos
Sass + Tai
Vite defau
Next.js de
Bootstrap

⚠ COMMON MISTAKES

- Thinking PostCSS replaces Sass — they're complementary; PostCSS transforms CSS output, Sass is a CSS authoring language
- Using Less in new projects — its development has slowed significantly; Sass offers far more features and community support
- Installing node-sass instead of sass — node-sass wraps deprecated LibSass; always install the sass package (Dart Sass)

✓ TIPS

- Use Sass for authoring + PostCSS for output — this combination gets you the best of both worlds
- PostCSS's preset-env lets you write native CSS nesting and custom media today with backward compatibility
- Migrate Less to Sass incrementally — the syntax is similar; variables, nesting, and mixins map closely

SUMMARY

- Sass — best authoring: variables, maps, modules, mixins, functions
- PostCSS — best transforms: autoprefixing, future CSS, custom plugins
- Less — legacy; avoid for new projects
- Common combo: Sass (authoring) + PostCSS (output pipeline)

Toolchain Integration Reference

Vite · Next.js · webpack · Astro · SvelteKit · Nuxt · Remix · configuration patterns

FRAMEWORK INTEGRATION PATTERNS

Every major build tool and framework handles CSS systems slightly differently. Understanding the integration points — where config goes, what gets auto-detected, and what needs explicit configuration — saves significant setup time. This sheet is a quick reference for the most common framework/tool combinations across the CSS systems covered in this binder.

INTEGRATION RULES OF THUMB

- PostCSS config is auto-detected from project root in Vite, Next.js, and Parcel
- Sass requires `npm install -D sass` — the runtime; no additional loader config in Vite
- CSS Modules work by naming files `*.module.css` in all major tools
- Tailwind requires `postcss.config.js` with the Tailwind plugin
- Component frameworks (Bootstrap/Bulma) can use CDN or npm + Sass override

VITE INTEGRATION

CSS SYSTEM	SETUP
Sass	npm i -D sass · auto-detected
CSS Modules	*.module.css · auto-detected
Tailwind	postcss.config.js + @tailwind directives
UnoCSS	unocss/vite plugin + virtual:uno.css import
styled-components	npm i styled-components · no config
PostCSS	postcss.config.js auto-detected
Less	npm i -D less · auto-detected
Open Props	npm i open-props · @import in CSS

NEXT.JS INTEGRATION

CSS SYSTEM	SETUP
CSS Modules	*.module.css · built-in
Sass	npm i -D sass · built-in after install
Tailwind	postcss.config.js · next.config.js no change
styled-components	next.config.js + Babel/SWC plugin
Emotion	next.config.js + emotion plugin
CSS Modules + Sass	*.module.scss · same as Sass
Global CSS	import in _app.js / layout.js
@layer	Works natively in all CSS files

OTHER FRAMEWORKS

FRAMEWORK	CSS SYSTEM	CONFIG
Astro	Tailwind	@astrojs/tailwi integration
Astro	Scoped CSS	Built-in (style in .astro)
SvelteKit	Tailwind	svelte-preproc postcss
SvelteKit	Sass	svelte-preproc sass option
Nuxt 3	Tailwind	@nuxtjs/tailwin module
Nuxt 3	UnoCSS	@unocss/nuxt module
Remix	CSS Modules	*.module.css bu in
Remix	Tailwind	postcss.config.j
Gatsby	Sass	gatsby-plugin-s

⚠ COMMON MISTAKES

- Adding PostCSS config after installing Tailwind but forgetting to restart the dev server — config changes require a restart
- In Next.js: importing global CSS in non-_app.js files — only _app.js or layout.js can import global stylesheets
- Styled-components SSR in Next.js without the Babel/SWC plugin — causes hydration mismatches and FOUC

✓ TIPS

- Check the official integration docs for each framework — they update frequently and often have one-command setup
- Vite's CSS preprocessing is synchronous and fast — prefer Vite for CSS-heavy projects where build speed matters
- Use CSS Modules as the default in server-component-heavy apps (Next.js App Router) — they have zero runtime overhead

SUMMARY

- Vite: sass + CSS Modules auto-detected; PostCSS via postcss.config.js
- Next.js: Sass + CSS Modules built-in; Tailwind via postcss.config.js
- CSS Modules work everywhere with *.module.css naming convention
- styled-components SSR requires explicit plugin/config in every framework

Migration & Interop Guide

MIGRATION STRATEGIES

Migrating between CSS systems works best as an **incremental strangler-fig**: the new system grows alongside the old one, replacing it component by component. CSS `@layer` enables clean coexistence by giving you explicit control over which system wins specificity battles during the transition period. Never do a big-bang rewrite.

MIGRATION PRINCIPLES

- Migrate incrementally, one component at a time
- Use `@layer` to control specificity between old and new
- Delete old code immediately as each component migrates
- Agree on a feature freeze for the old system before starting
- Add visual regression tests before migrating each component

BOOTSTRAP → TAILWIND

BOOTSTRAP CLASS	TAILWIND EQUIVALENT
`d-flex`	flex
`d-none d-md-block`	hidden md:block
`justify-content-between`	justify-between
`align-items-center`	items-center
`mt-3 / p-3`	mt-3 / p-3 (compatible scale)
`text-center`	text-center
`fw-bold`	font-bold
`rounded / shadow`	rounded / shadow
`container mx-auto`	container mx-auto
`col-md-6`	md:w-1/2
`btn btn-primary`	Custom @apply component class

SASS VARS → CSS CUSTOM PROPS

SASS	CSS CUSTOM PROPERTY
`$primary: #6366f1`	--color-primary: #6366f1
`color: $primary`	color: var(--color-primary)
`padding: $spacing-4`	padding: var(--spacing-4)
`Compile-time only`	Runtime; JS changeable
`No dark mode toggle`	Override in @media prefers-dark
`Sass map access`	--color-primary-500 naming
`@if $dark`	:root { --bg: #0f172a }

@IMPORT → @USE

OLD @IMPORT
`@import 'variables'`
`$primary (global)`
`@include flex-center`
`Runs on every @import`
`All variables global`
`Many @imports in barrel`

COEXISTENCE WITH @LAYER

GOAL	STRATEGY
`New code wins`	Put legacy in @layer legacy; new unlayered
`Legacy wins`	Put new code in @layer new; legacy unlayered
`Explicit order`	@layer legacy, new; later = higher specificity
`Full isolation`	Scope each system to a container class
`Tailwind + Bootstrap`	@layer { @import bootstrap }; Tailwind last

COMMON MIGRATION PATHS

FROM	TO	APPROACH
`Bootstrap`	Tailwind	Component-by-component; @layer coexistence
`Less`	Sass	Variable/mixin syntax similar; file-by-file
`Sass @import`	Sass @use	Add @use; fix namespace refs; remove @import
`Global CSS`	CSS Modules	Rename to .module.css on each component touch
`styled-components`	Vanilla Extract	Component-by-component; no coexistence issue
`Bootstrap jQuery`	Bootstrap 5	Remove jQuery; use data-bs-* attributes

MIGRATION CHECKLIST

STEP	ACTION
1	Audit current CSS: size, specificity, dead code
2	Define target system and coexistence strategy
3	Set up new system alongside old (no deletions yet)
4	Migrate highest-traffic components first
5	Add visual regression tests before/after each
6	Delete old code immediately as components migrate
7	Remove old system package when fully migrated
8	Update linting rules to enforce new system only

⚠ COMMON MISTAKES

- Big-bang rewrite — specificity conflicts between old and new break layouts unpredictably without coexistence strategy
- Migrating utilities first instead of layout components — layout impact validates the new system fastest
- Keeping old code "just in case" — migration debt compounds; both systems require maintenance as they slowly drift

✓ TIPS

- Use CSS `@layer` to give migrated components predictable specificity during coexistence period
- Migrate components with dedicated QA owners first — faster feedback loop on visual regressions
- Track migration progress as a percentage of components done — shared visibility accelerates the work

SUMMARY

- Incremental strangler-fig — new system grows alongside old
- CSS @layer — controls specificity between legacy and new during coexistence
- Delete old code immediately — migration debt compounds if orphaned
- Visual regression test every component before and after migration

Sheet 47

Quick Reference — Utility & Component Frameworks

Tailwind setup, layout, type & states · UnoCSS · Bootstrap grid, components & utilities · Bulma · Foundation

TAILWIND SETUP

ITEM	VALUE	#
v4 CSS entry	`@import "tailwindcss";`	01
v4 theme	`@theme { --color-brand: … }`	05
v4 Vite plugin	`@tailwindcss/vite`	01
v3 entry	`@tailwind base; components; utilities`	01
v3 config	`tailwind.config.js` + `content`	01
`@apply` / `@layer`	reuse utilities in CSS	05

TAILWIND LAYOUT

CLASS	DOES	#
`flex` / `grid` / `block`	display	02
`gap-4` / `p-4` / `m-2`	spacing (4 = 1rem)	02
`px-6` / `mt-auto`	axis / side spacing	02
`grid-cols-3` / `col-span-2`	grid tracks	02
`w-full` / `max-w-prose`	widths	02
`items-center` / `justify-between`	alignment	02

TAILWIND TYPE & STATES

CLASS	DOES	#
`text-sm` / `text-2xl`	font size	03
`font-bold` / `tracking-wide`	weight / spacing	03
`text-slate-700` / `bg-blue-500`	colors	03
`hover:` / `focus:` / `disabled:`	state variants	04
`md:` / `lg:`	breakpoints (mobile first)	04
`dark:`	dark mode variant	04
`w-[37%]`	arbitrary value	05

UNOCSS

ITEM	DOES	#
`uno.config.ts`	config file	06
`presetWind3()` / `presetWind4()`	Tailwind-compatible preset	06
`presetIcons()`	`i-mdi-home` icon classes	06
`presetAttributify()`	`<div p="4" text="red">`	07
`shortcuts: { btn: "…" }`	class aliases	07
`rules: [[/^m-(\d+)$/, …]]`	custom rules	08

BOOTSTRAP GRID

CLASS	DOES	#
`container` / `container-fluid`	page wrapper	09
`row` / `col`	grid row / auto column	09
`col-md-6`	6 of 12 from md up	09
`g-3` / `gx-2`	gutters	09
`offset-md-2` / `order-1`	offset / reorder	09
`sm md lg xl xxl`	breakpoints	09

BOOTSTRAP COMPONENTS

CLASS	DOES	#
`btn btn-primary`	button	10
`card` / `card-body`	card	10
`navbar navbar-expand-lg`	nav bar	10
`modal` + `data-bs-toggle`	modal dialog	10
`alert alert-warning`	alert	10
`form-control` / `is-invalid`	inputs / validation	12

BOOTSTRAP UTILITIES

CLASS	DOES	#
`d-flex` / `d-none d-md-block`	display	11
`p-3` / `mt-2` / `mx-auto`	spacing (0–5)	11
`text-center` / `fw-bold`	text	11
`bg-body-tertiary` / `text-muted`	color	11
`data-bs-theme="dark"`	color mode (5.3)	11
`$primary` + `@import`	Sass customization	13

BULMA

CLASS	DOES	#
`columns` / `column`	flex grid	14
`is-half` / `is-4`	column width	14
`button is-primary`	button + color	15
`box` / `card` / `notification`	elements	15
`is-large` / `has-text-centered`	modifiers	16
`--bulma-primary-h`	v1 CSS variables	16

FOUNDATION

CLASS	DOES	#
`grid-x` / `cell`	XY grid	17
`small-12 medium-6`	responsive cells	17
`grid-margin-x`	gutters	17
`button` / `callout`	components	18
`top-bar` / `reveal`	nav / modal	18
`_settings.scss`	Sass settings	19

⚠ COMMON MISTAKES

- Tailwind classes built by string concatenation — the scanner can't see them, so they're purged
- Mixing Tailwind v3 config habits into a v4 project (`@theme` replaces most of the JS config)
- Loading Bootstrap's JS twice, or forgetting `bootstrap.bundle` for dropdowns and popovers

✓ TIPS & BEST PRACTICES

- Keep full class names in source; map state to classes with an object
- Customize Bootstrap with Sass variables before `@import`, not overrides after
- Pick one framework per project — mixing grids causes specificity fights

QUICK SUMMARY

- Tailwind / UnoCSS: utilities composed in markup
- Bootstrap / Bulma / Foundation: components + grid classes
- All are mobile-first with breakpoint prefixes
- Theme via CSS variables or Sass settings

Sheet 48

Quick Reference — CSS-in-JS, Sass & Architecture

styled-components · CSS Modules · Sass syntax, mixins & modules · PostCSS plugins · Open Props tokens · ITCSS layers · picking a tool

STYLED-COMPONENTS

SYNTAX	DOES	#
`styled.button`…``	styled element	20
`${p => p.$primary && css`…`}`	prop-based style	21
`$prop (transient)`	not passed to the DOM	21
`styled(Link)`…``	extend a component	21
`<ThemeProvider theme={t}>`	theming	22
`createGlobalStyle`	global CSS	22
`.attrs({ type: "button" })`	default attributes	23

CSS MODULES

SYNTAX	DOES	#
`Button.module.css`	file naming	24
`import s from "./x.module.css"`	import class map	24
`className={s.primary}`	use a class	24
`:global(.x)`	opt out of scoping	24
`composes: base`	reuse another class	25
`@value brand: #f60`	shared values	25
`localsConvention: "camelCase"`	`s.myClass` access	26

SASS BASICS

SYNTAX	DOES	#
`$brand: #f60`	variable	28
`&:hover / &__elem`	parent selector	29
`.a { .b { } }`	nesting	29
`%placeholder + @extend`	shared rules	30
`@if / @each / @for`	control flow	30
`map.get($m, key)`	maps	28

SASS MIXINS & MODULES

SYNTAX	DOES	#
`@mixin m($x) { }`	define mixin	30
`@include m(2px)`	use mixin	30
`@function f($x) { @return … }`	function	30
`@use "tokens" as t`	load module (namespaced)	31
`@forward "src/colors"`	re-export	31
`@use "x" with ($a: 1)`	configure defaults	31
`@import`	deprecated — migrate to `@use`	31

POSTCSS

ITEM	DOES	#
`postcss.config.js`	plugin list	32
`autoprefixer`	vendor prefixes	33
`postcss-preset-env`	future CSS today	33
`postcss-nesting`	native nesting syntax	33
`cssnano`	minify	33
`postcss-import`	inline `@import`	33
`Declaration / Once hooks`	writing plugins	34

OPEN PROPS

TOKEN	EXAMPLE	#
`@import "open-props/style"`	all tokens	35
`--size-3 / --space`	spacing scale	36
`--font-size-2`	type scale	36
`--gray-7 / --blue-6`	color ramps	36
`--shadow-3 / --radius-2`	elevation / corners	36
`--ease-3 / --animation-fade-in`	motion	36
`normalize.min.css`	adaptive light / dark	37

ITCSS LAYERS

LAYER	HOLDS	#
`Settings`	variables, tokens	38
`Tools`	mixins, functions	38
`Generic`	reset / normalize	38
`Elements`	bare HTML tags	38
`Objects`	layout patterns `.o-`	39
`Components`	UI pieces `.c-`	39
`Utilities`	overrides `.u-`	39

PICKING A TOOL

NEED	REACH FOR	#
`Fast prototyping`	Tailwind / UnoCSS	41
`Prebuilt components`	Bootstrap / Bulma	42
`Styles beside React code`	styled-components	43
`Zero-runtime scoping`	CSS Modules	43
`Variables, mixins, loops`	Sass	44
`Big shared codebase`	ITCSS + tokens	40

⚠ COMMON MISTAKES

- Creating styled-components inside a render function — a new class every render
- Global selectors leaking out of a CSS Module without `:global` intent
- Still using Sass `@import`, which is deprecated and slated for removal

✓ TIPS & BEST PRACTICES

- Use transient `$props` so styling props don't reach the DOM
- Order PostCSS plugins: import → nesting / preset-env → autoprefixer → cssnano
- Map ITCSS layers to CSS `@layer` for predictable cascade order

QUICK SUMMARY

- CSS-in-JS: styles as components; Modules: scoped files
- Sass: `$vars`, nesting, `@mixin`, `@use`
- PostCSS: plugins transform standard CSS
- ITCSS: specificity rises layer by layer